

GPT + AI AUTOMATION

AI OUTSOURCING JOBS

20-DAY MASTERCLASS

200-page detailed lecture handbook | 60 minutes per day

- GPT & LLM Foundations
- Prompt Engineering & Structured Outputs
- OpenAI API + Function Calling
- n8n Business Automation
- RAG + Vector Search
- AI Agents + OpenAI Agents SDK
- LangGraph Orchestration
- Security, Evals & Deployment
- Upwork / LinkedIn / Client Delivery
- Full Capstone + 90-Day Roadmap

Professional training edition - August 2026

Verify current API models, pricing and platform limits before client production use.

How to Use This Handbook

This handbook is designed to be taught or self-studied one day at a time. Every day is a 60-minute lecture plus practical work. The nine pages for each day deliberately separate lecture delivery, engineering theory, implementation, troubleshooting, client application and portfolio evidence.

Recommended cadence: 45-60 minutes lecture, then 60-120 minutes of hands-on work outside the lecture. Senior capability comes from repeated implementation, debugging, evaluation and client delivery - not from reading alone.

Use test data and sandbox accounts while learning. Never place secrets in source control. For workflows that can send messages, move money, delete data, change permissions or affect people, require explicit policy checks and human approval until the system has been validated.

20-Day Course Map

Days 1-4: LLM application foundations, prompting, OpenAI API and tool calling.

Days 5-6: n8n workflow engineering and production reliability.

Days 7-8: RAG from fundamentals to permission-aware production systems.

Days 9-11: agents, OpenAI Agents SDK and stateful orchestration.

Days 12-14: business, document and multimodal automation.

Days 15-16: evals, security, privacy, deployment, observability and cost.

Days 17-18: discovery, architecture, Upwork/LinkedIn and proposals.

Days 19-20: production-style capstone, handoff and the 90-day continuation roadmap.

Core engineering skills: Python or TypeScript, REST APIs, JSON, webhooks, auth, Git, environment variables, logging and testing.

LLM skills: prompt design, structured outputs, tool calling, embeddings, RAG, agent design, memory, evals and guardrails.

Automation skills: n8n workflow design, branching, batching, retries, idempotency, API integrations and exception handling.

Consulting skills: discovery, scope, ROI, architecture, acceptance criteria, documentation, proposal writing, demos and maintenance.

Portfolio goal: by Day 20, have at least four demonstrable projects: API microservice, n8n workflow, RAG assistant and end-to-end capstone.

Recommended Tool Stack

Minimum: VS Code, Git/GitHub, Python 3.11+ or Node.js, Docker basics, Postman/Insomnia, OpenAI API account, n8n, and a simple SQL database.

RAG: start with a local/simple vector store for learning, then understand managed options such as pgvector/Pinecone-style services.

Agent frameworks: begin with the OpenAI Agents SDK for OpenAI-focused systems; learn LangGraph when you need explicit state graphs and complex orchestration.

Automation alternatives: Make/Zapier-style platforms are useful, but the engineering principles in this handbook focus on n8n because it exposes workflow logic, APIs and self-hosting concepts clearly.

Always verify current product names, model availability, pricing and limits in official documentation before production work.

1. Create a project folder and Git repository.
2. Create .env.example, but never commit real secrets.
3. Install the official OpenAI SDK for your language.
4. Run n8n locally or use a managed workspace.
5. Create a test database and mock data.
6. Configure an API testing client.
7. Add folders: /src, /workflows, /prompts, /evals, /docs, /tests.
8. Use a README with setup, architecture, environment variables and demo steps.
9. Add a simple issue log: problem, root cause, fix, prevention.
10. Treat every day as a portfolio artifact, not a disposable tutorial.

Prompts: store in versioned files or configuration; do not bury critical instructions across code.

Schemas: define explicit JSON structures for machine-consumed outputs.

Tools: use narrow permissions and descriptive names; validate every argument.

Logging: log IDs, timing, model/tool metadata and error categories; avoid unnecessary sensitive text.

Retries: retry transient failures with backoff; do not blindly retry validation or permission failures.

Testing: maintain deterministic unit tests plus AI eval cases.

Documentation: explain assumptions, dependencies, limits and manual recovery steps.

Security: separate read-only from write capabilities and place approval around high-impact actions.

1. Job brief or lead arrives.
2. Extract business outcome, systems, volume, constraints and deadlines.
3. Ask discovery questions.
4. Map current process and failure costs.
5. Design the smallest valuable phase.
6. Define architecture and acceptance criteria.
7. Build with sandbox data.
8. Test normal, edge and adversarial cases.
9. Demo against agreed workflow.
10. Deploy with monitoring and rollback.
11. Train the client and transfer credential ownership.
12. Offer maintenance or optimization based on real usage data.

A senior freelancer is paid for reducing uncertainty and operational risk, not just for connecting nodes.

Daily assessment: 4 interview questions + practical deliverable.
Weekly checkpoint after Days 5, 10, 15 and 20.

Portfolio artifacts:

- API integration microservice
- Production n8n workflow
- RAG knowledge assistant
- Agent orchestration demo
- Document automation workflow
- Capstone AI Operations Assistant

Evaluation rubric (100 points): correctness 25, reliability 20, security 15, testing/evals 15, documentation 10, architecture 10, client communication 5. A project is not "done" if the happy path works but failure handling, permissions and documentation are missing.

Table of Contents

The 20 daily lectures begin on page 11. Each day occupies exactly nine pages.

Day 1 p11 | Day 2 p20 | Day 3 p29 | Day 4 p38 | Day 5 p47
Day 6 p56 | Day 7 p65 | Day 8 p74 | Day 9 p83 | Day 10 p92
Day 11 p101 | Day 12 p110 | Day 13 p119 | Day 14 p128 | Day 15 p137
Day 16 p146 | Day 17 p155 | Day 18 p164 | Day 19 p173 | Day 20 p182

Appendices begin on page 191: Prompt Patterns, API Cheat Sheet, n8n Patterns, RAG Checklist, Agent Checklist, Security/Privacy, Client Discovery, Proposal Templates, Portfolio/Interview Rubric, References + 90-Day Roadmap.

ROLE TARGET

AI Automation Consultant / LLM Application Developer

WHY THIS DAY MATTERS

Clients rarely need a model in isolation; they need a dependable business system. Today you will connect the technical topic to architecture, failure handling and a portfolio-ready deliverable.

LEARNING OBJECTIVES

- Explain how modern LLM applications differ from traditional software.
- Map a client business problem into model, data, tools, workflow and human-review components.
- Understand tokens, context, inference, hallucination, determinism, latency and cost as engineering constraints.
- Identify common AI outsourcing job types and the skills each requires.

60-MINUTE DELIVERY PLAN

0-5 min: goals/job relevance; 5-15: core concepts; 15-25: architecture; 25-35: implementation; 35-50: guided lab; 50-56: debugging/security; 56-60: interview check + assignment.

END-OF-DAY EVIDENCE

Leave with implementation notes, a tested artifact, interview answers and a client-facing explanation of tradeoffs.

INSTRUCTOR PROMPTS

- Give one business example where this skill creates measurable value.
- Which part should remain deterministic code?
- What data or permissions make the design risky?
- How would you evaluate the result before release?

PRE-LECTURE PREP

Open the project repository, prepare sandbox credentials/test data, review the previous day and write one question the lab should answer.

5-MINUTE CHECK

Explain today's concept without buzzwords, sketch the architecture from memory, and name the most expensive likely failure.

CORE CONCEPTS - PART I

1. AI vs machine learning vs generative AI. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
2. Transformer and next-token prediction intuition. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
3. Tokens, context windows and message roles. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
4. Inference, temperature-style randomness and reproducibility. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.

TEACHING EMPHASIS

Do not memorize vocabulary. Translate each concept into a design choice and a measurable outcome.

CONCEPT DRILL

Complete: 'I would use this when ____; avoid it when ____; test it by ____.'

COMMON FAILURE PATTERN

Junior implementations often optimize the model call while ignoring validation, permissions, retries, data quality and human exception handling. Those surrounding controls usually determine production success.

SENIOR TAKEAWAY

The more probabilistic the component, the more explicit its measurement and fallback strategy must be.

CORE CONCEPTS - PART II

5. Hallucination vs grounded generation. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
6. Model selection: quality, latency, cost, modalities. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
7. AI application architecture. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
8. Human-in-the-loop and escalation. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

SENIOR-LEVEL CONNECTION

Every probabilistic decision needs validation/evaluation; every side effect needs authorization; every integration needs a failure strategy; every quality claim needs evidence.

TRADEOFF EXERCISE

Compare a fast/cheap implementation with a high-reliability implementation. Discuss cost, latency, complexity, accuracy and failure impact.

CLIENT TRANSLATION

Sell the changed business process, not the technical term: what manual step changes, what evidence is used, what becomes faster, and what a human still approves.

REVIEW QUESTION

If this component were wrong for one week, how would the team notice? If the answer is 'they would not', add observability or an eval checkpoint.

ARCHITECTURE WALKTHROUGH

Reference flow:

User/UI -> Application backend -> Prompt/instructions -> LLM API -> Knowledge sources -> Tools/APIs -> Business rules -> Human approval -> Logs/evaluations

1. User/UI: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
2. Application backend: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
3. Prompt/instructions: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
4. LLM API: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
5. Knowledge sources: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
6. Tools/APIs: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
7. Business rules: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
8. Human approval: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
9. Logs/evaluations: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.

DESIGN REVIEW QUESTIONS

- Where is untrusted input introduced?
- Where are secrets or sensitive data present?
- Which step is probabilistic?
- Which step changes a system of record?
- What is the human exception path?
- What proves success?

FAILURE-MODE WALKTHROUGH

Test mentally: model timeout, stale data, 429, expired credential, duplicate webhook, unauthorized request. Add a graceful path for relevant failures.

BOUNDARY RULE

Never pass probabilistic output directly into a high-impact side effect. Insert schema validation, business rules, authorization and, where needed, human approval.

DELIVERABLE

Create a technical diagram and a simpler client diagram.

IMPLEMENTATION WALKTHROUGH

```
# Conceptual request
request = {
  "instructions": "Act as a support triage assistant",
  "input": customer_message,
  "context": policy_snippets,
  "tools": [lookup_order, create_ticket]
}
# A production system validates outputs before acting.
```

HOW TO READ THIS EXAMPLE

1. Identify component contracts. 2. Separate configuration/secrets. 3. Validate input. 4. Validate model/tool output. 5. Add structured errors and correlation IDs. 6. Prefer explicit schemas over parsing prose.

PRODUCTION EXTENSIONS

Add timeouts, transient retries, rate-limit handling, test fixtures, logging, environment config and a small eval dataset.

TEST PLAN

- happy path
- malformed/missing fields
- timeout / rate-limit
- invalid structured output
- duplicate request
- unauthorized action
- downstream rejection

CODE-REVIEW QUESTIONS

Where can exceptions escape? Which values are trusted? Which calls create side effects? Are secrets loaded safely? Can one request be traced end-to-end? Can prompts/business rules change independently?

OUTSOURCING STANDARD

Deliver code plus config template, setup steps, sample data, tests/evals and known limitations. 'Works on my machine' is not a handoff.

GUIDED LAB - PART I

LAB GOAL

Create a one-page architecture for an AI customer-support triage system. Define inputs, outputs, failure cases, data sources, tools, approvals and success metrics.

STEP 1 - DEFINE ACCEPTANCE

Write 3-5 observable success conditions before building, including one must-not-happen condition.

STEP 2 - PREPARE TEST DATA

Create normal, edge and malformed examples. Use sandbox/synthetic data.

STEP 3 - BUILD A VERTICAL SLICE

Implement one end-to-end path from trigger/input to validated output. Keep risky side effects mocked or sandboxed first.

STEP 4 - ADD OBSERVABILITY

Record request/workflow ID, timing, decision result, validation result and error category.

STEP 5 - INSPECT INTERMEDIATES

Run manually and inspect each boundary once before adding complexity.

STEP 6 - CAPTURE BASELINE

Record first success rate, latency and estimated cost/task.

STEP 7 - WRITE ACCEPTANCE TESTS

Turn success conditions into repeatable tests/evals, including one case that must escalate.

LAB DELIVERABLE

Commit code/workflow export, sample payloads, architecture image, README and test results.

GUIDED LAB - PART II: HARDENING & DEBUGGING

Test empty input, unexpected type, missing credential, provider timeout, rate limit, low-confidence AI output, invalid structured output, duplicate trigger and downstream API rejection.

DEBUGGING METHOD

1. Reproduce minimally.
2. Locate the failing boundary.
3. Inspect raw input/output safely.
4. Classify: validation, model, network, auth, permission, rate limit, provider, business rule or data.
5. Fix root cause.
6. Add regression test/eval.
7. Document symptom and recovery.

ROOT-CAUSE VS PATCH

Do not add arbitrary delays, broad retries or larger prompts without evidence. Name the root cause and the prevention mechanism.

RECOVERY DESIGN

For each terminal failure choose: retry later, review queue, compensate/rollback, alert operator, or safely discard.

CLIENT-READY FINISH

README sections: Known Limitations and Operations / Recovery.

PORTFOLIO TIP

Include one 'What went wrong' story. A well-explained bug and fix demonstrates seniority better than a perfect demo.

CLIENT SCENARIO

A small e-commerce brand wants to reduce repetitive support work. Convert the vague request "add AI to support" into a scoped first phase that can be delivered in one week.

HOW A SENIOR FREELANCER RESPONDS

Restate the business outcome. Ask about process, volume, systems, sensitive data, permissions, exceptions and cost of mistakes. Propose the smallest phase that proves value. Define AI vs deterministic steps, approvals, acceptance criteria and handoff.

SECURITY / RELIABILITY REVIEW

Check least-privilege credentials, input/output validation, prompt injection exposure, privacy, idempotency, retries, escalation, audit logs and rollback.

SENIOR INTERVIEW PRACTICE

Q1. Why is an LLM not a database?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q2. What causes hallucinations and how do you reduce business risk?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q3. When should you use deterministic workflow logic instead of an agent?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q4. How do latency, cost and accuracy trade off?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

DISCOVERY QUESTIONS

What happens today? What is volume and error cost? Which APIs are mandatory? Which data is sensitive? Which actions need approval? What metric defines a successful pilot?

PROPOSAL ANGLE

Offer a first milestone that reduces the highest uncertainty: API feasibility, retrieval quality, classification accuracy or workflow reliability.

COMMERCIAL NOTE

Estimate from scope/dependencies/risk. Never promise 100% accuracy or full autonomy when acceptable error handling is undefined.

HOMEWORK & PORTFOLIO OUTPUT

- Write a 500-word explanation of an LLM application architecture for a non-technical client.
- Create a GitHub repository named ai-automation-masterclass and add Day 1 architecture notes.
- Collect five AI automation job descriptions and classify each by prompt/API/RAG/agent/workflow requirements.

PORTFOLIO CHECKLIST

- problem statement
- architecture diagram
- setup instructions
- example input/output
- tests or eval cases
- error handling
- security/permissions note
- known limitations
- demo steps

SUBMISSION RUBRIC

Correctness 30% | Reliability 20% | Security 15% | Tests/evals 15% | Documentation 10% | Architecture reasoning 10%.

CLIENT-FACING SUMMARY

Write five sentences: problem, automated flow, where AI is used, how risk is controlled, and expected measurable result. Reuse this in LinkedIn/portfolio/Upwork material.

SELF-ASSESSMENT

Can you justify the architecture, reproduce a failure, diagnose it, and explain scope/success to a client?

NEXT-DAY BRIDGE

Record one unresolved technical question and one business question.

REFERENCE HABIT

Check current official docs for SDK syntax, model/tool availability, limits and platform behavior before production.

ROLE TARGET

Prompt Engineer / AI Workflow Designer

WHY THIS DAY MATTERS

Clients rarely need a model in isolation; they need a dependable business system. Today you will connect the technical topic to architecture, failure handling and a portfolio-ready deliverable.

LEARNING OBJECTIVES

- Design prompts as maintainable software assets rather than one-off text.
- Use role, task, context, constraints, examples and output contracts.
- Choose between free-form generation and schema-constrained outputs.
- Create reusable prompt test cases and failure-mode checks.

60-MINUTE DELIVERY PLAN

0-5 min: goals/job relevance; 5-15: core concepts; 15-25: architecture; 25-35: implementation; 35-50: guided lab; 50-56: debugging/security; 56-60: interview check + assignment.

END-OF-DAY EVIDENCE

Leave with implementation notes, a tested artifact, interview answers and a client-facing explanation of tradeoffs.

INSTRUCTOR PROMPTS

- Give one business example where this skill creates measurable value.
- Which part should remain deterministic code?
- What data or permissions make the design risky?
- How would you evaluate the result before release?

PRE-LECTURE PREP

Open the project repository, prepare sandbox credentials/test data, review the previous day and write one question the lab should answer.

5-MINUTE CHECK

Explain today's concept without buzzwords, sketch the architecture from memory, and name the most expensive likely failure.

CORE CONCEPTS - PART I

1. Instruction hierarchy. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
2. System/developer/user intent separation. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
3. Task decomposition. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
4. Context packaging. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
5. Few-shot examples. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.

TEACHING EMPHASIS

Do not memorize vocabulary. Translate each concept into a design choice and a measurable outcome.

CONCEPT DRILL

Complete: 'I would use this when ____; avoid it when ____; test it by ____.'

COMMON FAILURE PATTERN

Junior implementations often optimize the model call while ignoring validation, permissions, retries, data quality and human exception handling. Those surrounding controls usually determine production success.

SENIOR TAKEAWAY

The more probabilistic the component, the more explicit its measurement and fallback strategy must be.

CORE CONCEPTS - PART II

6. Output contracts and JSON schemas. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
7. Prompt injection awareness. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
8. Prompt versioning. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
9. Prompt evaluation datasets. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

SENIOR-LEVEL CONNECTION

Every probabilistic decision needs validation/evaluation; every side effect needs authorization; every integration needs a failure strategy; every quality claim needs evidence.

TRADEOFF EXERCISE

Compare a fast/cheap implementation with a high-reliability implementation. Discuss cost, latency, complexity, accuracy and failure impact.

CLIENT TRANSLATION

Sell the changed business process, not the technical term: what manual step changes, what evidence is used, what becomes faster, and what a human still approves.

REVIEW QUESTION

If this component were wrong for one week, how would the team notice? If the answer is 'they would not', add observability or an eval checkpoint.

ARCHITECTURE WALKTHROUGH

Reference flow:

Business requirement -> Prompt template -> Input normalization -> Context assembly -> Model call -> Schema validation -> Business-rule validation -> Retry/fallback -> Human review

1. Business requirement: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
2. Prompt template: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
3. Input normalization: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
4. Context assembly: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
5. Model call: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
6. Schema validation: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
7. Business-rule validation: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
8. Retry/fallback: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
9. Human review: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.

DESIGN REVIEW QUESTIONS

- Where is untrusted input introduced?
- Where are secrets or sensitive data present?
- Which step is probabilistic?
- Which step changes a system of record?
- What is the human exception path?
- What proves success?

FAILURE-MODE WALKTHROUGH

Test mentally: model timeout, stale data, 429, expired credential, duplicate webhook, unauthorized request. Add a graceful path for relevant failures.

BOUNDARY RULE

Never pass probabilistic output directly into a high-impact side effect. Insert schema validation, business rules, authorization and, where needed, human approval.

DELIVERABLE

Create a technical diagram and a simpler client diagram.

IMPLEMENTATION WALKTHROUGH

PROMPT = """

You classify inbound leads.

Return ONLY fields matching the required schema.

Rules:

- Never invent budget or company size.
- Use null when evidence is missing.
- Escalate if confidence < 0.75.

Input: {lead_text}

"""

HOW TO READ THIS EXAMPLE

1. Identify component contracts. 2. Separate configuration/secrets. 3. Validate input. 4. Validate model/tool output. 5. Add structured errors and correlation IDs. 6. Prefer explicit schemas over parsing prose.

PRODUCTION EXTENSIONS

Add timeouts, transient retries, rate-limit handling, test fixtures, logging, environment config and a small eval dataset.

TEST PLAN

- happy path
- malformed/missing fields
- timeout / rate-limit
- invalid structured output
- duplicate request
- unauthorized action
- downstream rejection

CODE-REVIEW QUESTIONS

Where can exceptions escape? Which values are trusted? Which calls create side effects? Are secrets loaded safely? Can one request be traced end-to-end? Can prompts/business rules change independently?

OUTSOURCING STANDARD

Deliver code plus config template, setup steps, sample data, tests/evals and known limitations. 'Works on my machine' is not a handoff.

GUIDED LAB - PART I

LAB GOAL

Build three prompt versions for lead qualification. Test them against at least ten examples including missing data, conflicting details, prompt injection, and ambiguous requests.

STEP 1 - DEFINE ACCEPTANCE

Write 3-5 observable success conditions before building, including one must-not-happen condition.

STEP 2 - PREPARE TEST DATA

Create normal, edge and malformed examples. Use sandbox/synthetic data.

STEP 3 - BUILD A VERTICAL SLICE

Implement one end-to-end path from trigger/input to validated output. Keep risky side effects mocked or sandboxed first.

STEP 4 - ADD OBSERVABILITY

Record request/workflow ID, timing, decision result, validation result and error category.

STEP 5 - INSPECT INTERMEDIATES

Run manually and inspect each boundary once before adding complexity.

STEP 6 - CAPTURE BASELINE

Record first success rate, latency and estimated cost/task.

STEP 7 - WRITE ACCEPTANCE TESTS

Turn success conditions into repeatable tests/evals, including one case that must escalate.

LAB DELIVERABLE

Commit code/workflow export, sample payloads, architecture image, README and test results.

GUIDED LAB - PART II: HARDENING & DEBUGGING

Test empty input, unexpected type, missing credential, provider timeout, rate limit, low-confidence AI output, invalid structured output, duplicate trigger and downstream API rejection.

DEBUGGING METHOD

1. Reproduce minimally.
2. Locate the failing boundary.
3. Inspect raw input/output safely.
4. Classify: validation, model, network, auth, permission, rate limit, provider, business rule or data.
5. Fix root cause.
6. Add regression test/eval.
7. Document symptom and recovery.

ROOT-CAUSE VS PATCH

Do not add arbitrary delays, broad retries or larger prompts without evidence. Name the root cause and the prevention mechanism.

RECOVERY DESIGN

For each terminal failure choose: retry later, review queue, compensate/rollback, alert operator, or safely discard.

CLIENT-READY FINISH

README sections: Known Limitations and Operations / Recovery.

PORTFOLIO TIP

Include one 'What went wrong' story. A well-explained bug and fix demonstrates seniority better than a perfect demo.

CLIENT SCENARIO

A sales team asks for an AI that labels leads Hot/Warm/Cold. Define the classification rubric, evidence requirements, confidence policy and JSON output contract before choosing a model.

HOW A SENIOR FREELANCER RESPONDS

Restate the business outcome. Ask about process, volume, systems, sensitive data, permissions, exceptions and cost of mistakes. Propose the smallest phase that proves value. Define AI vs deterministic steps, approvals, acceptance criteria and handoff.

SECURITY / RELIABILITY REVIEW

Check least-privilege credentials, input/output validation, prompt injection exposure, privacy, idempotency, retries, escalation, audit logs and rollback.

SENIOR INTERVIEW PRACTICE

Q1. What makes a prompt production-ready?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q2. Why are examples useful, and when can they hurt?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q3. How do you prevent a model from silently inventing fields?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q4. How do you regression-test prompts?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

DISCOVERY QUESTIONS

What happens today? What is volume and error cost? Which APIs are mandatory? Which data is sensitive? Which actions need approval? What metric defines a successful pilot?

PROPOSAL ANGLE

Offer a first milestone that reduces the highest uncertainty: API feasibility, retrieval quality, classification accuracy or workflow reliability.

COMMERCIAL NOTE

Estimate from scope/dependencies/risk. Never promise 100% accuracy or full autonomy when acceptable error handling is undefined.

HOMEWORK & PORTFOLIO OUTPUT

- Create a prompt library with version numbers and test cases.
- Write a schema for extracting contact, intent, urgency, budget and next action from a lead email.
- Document five prompt failure modes and mitigations.

PORTFOLIO CHECKLIST

- problem statement
- architecture diagram
- setup instructions
- example input/output
- tests or eval cases
- error handling
- security/permissions note
- known limitations
- demo steps

SUBMISSION RUBRIC

Correctness 30% | Reliability 20% | Security 15% | Tests/evals 15% | Documentation 10% | Architecture reasoning 10%.

CLIENT-FACING SUMMARY

Write five sentences: problem, automated flow, where AI is used, how risk is controlled, and expected measurable result. Reuse this in LinkedIn/portfolio/Upwork material.

SELF-ASSESSMENT

Can you justify the architecture, reproduce a failure, diagnose it, and explain scope/success to a client?

NEXT-DAY BRIDGE

Record one unresolved technical question and one business question.

REFERENCE HABIT

Check current official docs for SDK syntax, model/tool availability, limits and platform behavior before production.

ROLE TARGET

LLM API Integration Developer

WHY THIS DAY MATTERS

Clients rarely need a model in isolation; they need a dependable business system. Today you will connect the technical topic to architecture, failure handling and a portfolio-ready deliverable.

LEARNING OBJECTIVES

- Create server-side API integrations using environment variables and official SDKs.
- Understand request/response lifecycle, streaming, errors and retries.
- Separate application configuration from secrets.
- Build a simple AI service endpoint suitable for outsourcing projects.

60-MINUTE DELIVERY PLAN

0-5 min: goals/job relevance; 5-15: core concepts; 15-25: architecture; 25-35: implementation; 35-50: guided lab; 50-56: debugging/security; 56-60: interview check + assignment.

END-OF-DAY EVIDENCE

Leave with implementation notes, a tested artifact, interview answers and a client-facing explanation of tradeoffs.

INSTRUCTOR PROMPTS

- Give one business example where this skill creates measurable value.
- Which part should remain deterministic code?
- What data or permissions make the design risky?
- How would you evaluate the result before release?

PRE-LECTURE PREP

Open the project repository, prepare sandbox credentials/test data, review the previous day and write one question the lab should answer.

5-MINUTE CHECK

Explain today's concept without buzzwords, sketch the architecture from memory, and name the most expensive likely failure.

CORE CONCEPTS - PART I

1. API keys and server-side secrets. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
2. Responses API mental model. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
3. Input messages and instructions. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
4. Streaming vs non-streaming. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
5. Timeouts and retries. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.

TEACHING EMPHASIS

Do not memorize vocabulary. Translate each concept into a design choice and a measurable outcome.

CONCEPT DRILL

Complete: 'I would use this when ____; avoid it when ____; test it by ____.'

COMMON FAILURE PATTERN

Junior implementations often optimize the model call while ignoring validation, permissions, retries, data quality and human exception handling. Those surrounding controls usually determine production success.

SENIOR TAKEAWAY

The more probabilistic the component, the more explicit its measurement and fallback strategy must be.

CORE CONCEPTS - PART II

6. Rate limits. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
7. Token/cost tracking. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
8. Async execution. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
9. Logging request metadata safely. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

SENIOR-LEVEL CONNECTION

Every probabilistic decision needs validation/evaluation; every side effect needs authorization; every integration needs a failure strategy; every quality claim needs evidence.

TRADEOFF EXERCISE

Compare a fast/cheap implementation with a high-reliability implementation. Discuss cost, latency, complexity, accuracy and failure impact.

CLIENT TRANSLATION

Sell the changed business process, not the technical term: what manual step changes, what evidence is used, what becomes faster, and what a human still approves.

REVIEW QUESTION

If this component were wrong for one week, how would the team notice? If the answer is 'they would not', add observability or an eval checkpoint.

ARCHITECTURE WALKTHROUGH

Reference flow:

Web/mobile client -> Backend endpoint -> Input validation -> OpenAI SDK -> Model response -> Output validation -> Application response -> Observability

1. Web/mobile client: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
2. Backend endpoint: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
3. Input validation: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
4. OpenAI SDK: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
5. Model response: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
6. Output validation: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
7. Application response: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
8. Observability: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.

DESIGN REVIEW QUESTIONS

- Where is untrusted input introduced?
- Where are secrets or sensitive data present?
- Which step is probabilistic?
- Which step changes a system of record?
- What is the human exception path?
- What proves success?

FAILURE-MODE WALKTHROUGH

Test mentally: model timeout, stale data, 429, expired credential, duplicate webhook, unauthorized request. Add a graceful path for relevant failures.

BOUNDARY RULE

Never pass probabilistic output directly into a high-impact side effect. Insert schema validation, business rules, authorization and, where needed, human approval.

DELIVERABLE

Create a technical diagram and a simpler client diagram.

IMPLEMENTATION WALKTHROUGH

```
from openai import OpenAI
import os
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])
resp = client.responses.create(
    model=os.environ.get("OPENAI_MODEL"),
    instructions="Summarize the customer request in 3 bullets.",
    input="Customer: " + message,
)
print(resp.output_text)
```

HOW TO READ THIS EXAMPLE

1. Identify component contracts. 2. Separate configuration/secrets. 3. Validate input. 4. Validate model/tool output. 5. Add structured errors and correlation IDs. 6. Prefer explicit schemas over parsing prose.

PRODUCTION EXTENSIONS

Add timeouts, transient retries, rate-limit handling, test fixtures, logging, environment config and a small eval dataset.

TEST PLAN

- happy path
- malformed/missing fields
- timeout / rate-limit
- invalid structured output
- duplicate request
- unauthorized action
- downstream rejection

CODE-REVIEW QUESTIONS

Where can exceptions escape? Which values are trusted? Which calls create side effects? Are secrets loaded safely? Can one request be traced end-to-end? Can prompts/business rules change independently?

OUTSOURCING STANDARD

Deliver code plus config template, setup steps, sample data, tests/evals and known limitations. 'Works on my machine' is not a handoff.

GUIDED LAB - PART I

LAB GOAL

Build a small Python or Node.js service with POST /summarize. Add input length validation, environment-variable configuration, structured error responses and basic logging.

STEP 1 - DEFINE ACCEPTANCE

Write 3-5 observable success conditions before building, including one must-not-happen condition.

STEP 2 - PREPARE TEST DATA

Create normal, edge and malformed examples. Use sandbox/synthetic data.

STEP 3 - BUILD A VERTICAL SLICE

Implement one end-to-end path from trigger/input to validated output. Keep risky side effects mocked or sandboxed first.

STEP 4 - ADD OBSERVABILITY

Record request/workflow ID, timing, decision result, validation result and error category.

STEP 5 - INSPECT INTERMEDIATES

Run manually and inspect each boundary once before adding complexity.

STEP 6 - CAPTURE BASELINE

Record first success rate, latency and estimated cost/task.

STEP 7 - WRITE ACCEPTANCE TESTS

Turn success conditions into repeatable tests/evals, including one case that must escalate.

LAB DELIVERABLE

Commit code/workflow export, sample payloads, architecture image, README and test results.

GUIDED LAB - PART II: HARDENING & DEBUGGING

Test empty input, unexpected type, missing credential, provider timeout, rate limit, low-confidence AI output, invalid structured output, duplicate trigger and downstream API rejection.

DEBUGGING METHOD

1. Reproduce minimally.
2. Locate the failing boundary.
3. Inspect raw input/output safely.
4. Classify: validation, model, network, auth, permission, rate limit, provider, business rule or data.
5. Fix root cause.
6. Add regression test/eval.
7. Document symptom and recovery.

ROOT-CAUSE VS PATCH

Do not add arbitrary delays, broad retries or larger prompts without evidence. Name the root cause and the prevention mechanism.

RECOVERY DESIGN

For each terminal failure choose: retry later, review queue, compensate/rollback, alert operator, or safely discard.

CLIENT-READY FINISH

README sections: Known Limitations and Operations / Recovery.

PORTFOLIO TIP

Include one 'What went wrong' story. A well-explained bug and fix demonstrates seniority better than a perfect demo.

CLIENT SCENARIO

A legal-operations team wants document summaries behind an internal API. Explain why the API key stays on the server, how data flows, and what must be logged without exposing sensitive text.

HOW A SENIOR FREELANCER RESPONDS

Restate the business outcome. Ask about process, volume, systems, sensitive data, permissions, exceptions and cost of mistakes. Propose the smallest phase that proves value. Define AI vs deterministic steps, approvals, acceptance criteria and handoff.

SECURITY / RELIABILITY REVIEW

Check least-privilege credentials, input/output validation, prompt injection exposure, privacy, idempotency, retries, escalation, audit logs and rollback.

SENIOR INTERVIEW PRACTICE

Q1. Why should API keys never be shipped in frontend code?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q2. When is streaming useful?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q3. How should you handle transient API failures?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q4. What should be included in LLM request observability?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

DISCOVERY QUESTIONS

What happens today? What is volume and error cost? Which APIs are mandatory? Which data is sensitive? Which actions need approval? What metric defines a successful pilot?

PROPOSAL ANGLE

Offer a first milestone that reduces the highest uncertainty: API feasibility, retrieval quality, classification accuracy or workflow reliability.

COMMERCIAL NOTE

Estimate from scope/dependencies/risk. Never promise 100% accuracy or full autonomy when acceptable error handling is undefined.

HOMEWORK & PORTFOLIO OUTPUT

- Implement the same request in Python and JavaScript.
- Add exponential backoff for retryable failures.
- Write README setup instructions that another developer can follow in under 10 minutes.

PORTFOLIO CHECKLIST

- problem statement
- architecture diagram
- setup instructions
- example input/output
- tests or eval cases
- error handling
- security/permissions note
- known limitations
- demo steps

SUBMISSION RUBRIC

Correctness 30% | Reliability 20% | Security 15% | Tests/evals 15% | Documentation 10% | Architecture reasoning 10%.

CLIENT-FACING SUMMARY

Write five sentences: problem, automated flow, where AI is used, how risk is controlled, and expected measurable result. Reuse this in LinkedIn/portfolio/Upwork material.

SELF-ASSESSMENT

Can you justify the architecture, reproduce a failure, diagnose it, and explain scope/success to a client?

NEXT-DAY BRIDGE

Record one unresolved technical question and one business question.

REFERENCE HABIT

Check current official docs for SDK syntax, model/tool availability, limits and platform behavior before production.

ROLE TARGET

AI Agent Integration Engineer

WHY THIS DAY MATTERS

Clients rarely need a model in isolation; they need a dependable business system. Today you will connect the technical topic to architecture, failure handling and a portfolio-ready deliverable.

LEARNING OBJECTIVES

- Connect models to application functions using typed tool definitions.
- Design safe tool inputs and outputs.
- Separate model decision-making from privileged execution.
- Implement approval and validation boundaries for side effects.

60-MINUTE DELIVERY PLAN

0-5 min: goals/job relevance; 5-15: core concepts; 15-25: architecture; 25-35: implementation; 35-50: guided lab; 50-56: debugging/security; 56-60: interview check + assignment.

END-OF-DAY EVIDENCE

Leave with implementation notes, a tested artifact, interview answers and a client-facing explanation of tradeoffs.

INSTRUCTOR PROMPTS

- Give one business example where this skill creates measurable value.
- Which part should remain deterministic code?
- What data or permissions make the design risky?
- How would you evaluate the result before release?

PRE-LECTURE PREP

Open the project repository, prepare sandbox credentials/test data, review the previous day and write one question the lab should answer.

5-MINUTE CHECK

Explain today's concept without buzzwords, sketch the architecture from memory, and name the most expensive likely failure.

CORE CONCEPTS - PART I

1. Function/tool calling. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
2. JSON-schema tool definitions. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
3. Tool selection. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
4. Tool-result loop. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
5. Idempotency. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.

TEACHING EMPHASIS

Do not memorize vocabulary. Translate each concept into a design choice and a measurable outcome.

CONCEPT DRILL

Complete: 'I would use this when ____; avoid it when ____; test it by ____.'

COMMON FAILURE PATTERN

Junior implementations often optimize the model call while ignoring validation, permissions, retries, data quality and human exception handling. Those surrounding controls usually determine production success.

SENIOR TAKEAWAY

The more probabilistic the component, the more explicit its measurement and fallback strategy must be.

CORE CONCEPTS - PART II

6. Authentication to third-party APIs. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
7. Read vs write tools. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
8. Side-effect safety. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
9. Approval gates. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
10. Least privilege. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

SENIOR-LEVEL CONNECTION

Every probabilistic decision needs validation/evaluation; every side effect needs authorization; every integration needs a failure strategy; every quality claim needs evidence.

TRADEOFF EXERCISE

Compare a fast/cheap implementation with a high-reliability implementation. Discuss cost, latency, complexity, accuracy and failure impact.

CLIENT TRANSLATION

Sell the changed business process, not the technical term: what manual step changes, what evidence is used, what becomes faster, and what a human still approves.

REVIEW QUESTION

If this component were wrong for one week, how would the team notice? If the answer is 'they would not', add observability or an eval checkpoint.

ARCHITECTURE WALKTHROUGH

Reference flow:

User request -> Model decides tool -> Schema validation -> Policy check -> Tool/API execution -> Tool result -> Model synthesis -> Audit log

1. User request: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
2. Model decides tool: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
3. Schema validation: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
4. Policy check: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
5. Tool/API execution: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
6. Tool result: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
7. Model synthesis: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
8. Audit log: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.

DESIGN REVIEW QUESTIONS

- Where is untrusted input introduced?
- Where are secrets or sensitive data present?
- Which step is probabilistic?
- Which step changes a system of record?
- What is the human exception path?
- What proves success?

FAILURE-MODE WALKTHROUGH

Test mentally: model timeout, stale data, 429, expired credential, duplicate webhook, unauthorized request. Add a graceful path for relevant failures.

BOUNDARY RULE

Never pass probabilistic output directly into a high-impact side effect. Insert schema validation, business rules, authorization and, where needed, human approval.

DELIVERABLE

Create a technical diagram and a simpler client diagram.

IMPLEMENTATION WALKTHROUGH

```
def lookup_order(order_id: str):  
    # Read-only tool: validate ID, call commerce API, return minimal fields  
    ...  
  
def refund_order(order_id: str, amount: float):  
    # Write tool: require policy checks + human approval  
    ...  
  
# Never let free-form model text become an unchecked API request.
```

HOW TO READ THIS EXAMPLE

1. Identify component contracts. 2. Separate configuration/secrets. 3. Validate input. 4. Validate model/tool output. 5. Add structured errors and correlation IDs. 6. Prefer explicit schemas over parsing prose.

PRODUCTION EXTENSIONS

Add timeouts, transient retries, rate-limit handling, test fixtures, logging, environment config and a small eval dataset.

TEST PLAN

- happy path
- malformed/missing fields
- timeout / rate-limit
- invalid structured output
- duplicate request
- unauthorized action
- downstream rejection

CODE-REVIEW QUESTIONS

Where can exceptions escape? Which values are trusted? Which calls create side effects? Are secrets loaded safely? Can one request be traced end-to-end? Can prompts/business rules change independently?

OUTSOURCING STANDARD

Deliver code plus config template, setup steps, sample data, tests/evals and known limitations. 'Works on my machine' is not a handoff.

GUIDED LAB - PART I

LAB GOAL

Create two read-only tools (customer lookup, order lookup) and one write tool (create ticket). Add argument validation and a manual approval requirement for any destructive or financial action.

STEP 1 - DEFINE ACCEPTANCE

Write 3-5 observable success conditions before building, including one must-not-happen condition.

STEP 2 - PREPARE TEST DATA

Create normal, edge and malformed examples. Use sandbox/synthetic data.

STEP 3 - BUILD A VERTICAL SLICE

Implement one end-to-end path from trigger/input to validated output. Keep risky side effects mocked or sandboxed first.

STEP 4 - ADD OBSERVABILITY

Record request/workflow ID, timing, decision result, validation result and error category.

STEP 5 - INSPECT INTERMEDIATES

Run manually and inspect each boundary once before adding complexity.

STEP 6 - CAPTURE BASELINE

Record first success rate, latency and estimated cost/task.

STEP 7 - WRITE ACCEPTANCE TESTS

Turn success conditions into repeatable tests/evals, including one case that must escalate.

LAB DELIVERABLE

Commit code/workflow export, sample payloads, architecture image, README and test results.

GUIDED LAB - PART II: HARDENING & DEBUGGING

Test empty input, unexpected type, missing credential, provider timeout, rate limit, low-confidence AI output, invalid structured output, duplicate trigger and downstream API rejection.

DEBUGGING METHOD

1. Reproduce minimally.
2. Locate the failing boundary.
3. Inspect raw input/output safely.
4. Classify: validation, model, network, auth, permission, rate limit, provider, business rule or data.
5. Fix root cause.
6. Add regression test/eval.
7. Document symptom and recovery.

ROOT-CAUSE VS PATCH

Do not add arbitrary delays, broad retries or larger prompts without evidence. Name the root cause and the prevention mechanism.

RECOVERY DESIGN

For each terminal failure choose: retry later, review queue, compensate/rollback, alert operator, or safely discard.

CLIENT-READY FINISH

README sections: Known Limitations and Operations / Recovery.

PORTFOLIO TIP

Include one 'What went wrong' story. A well-explained bug and fix demonstrates seniority better than a perfect demo.

CLIENT SCENARIO

A client asks for an AI assistant that can refund orders. Design a safer phased scope: first recommend refund decisions, then require approval, and only later consider bounded automation.

HOW A SENIOR FREELANCER RESPONDS

Restate the business outcome. Ask about process, volume, systems, sensitive data, permissions, exceptions and cost of mistakes. Propose the smallest phase that proves value. Define AI vs deterministic steps, approvals, acceptance criteria and handoff.

SECURITY / RELIABILITY REVIEW

Check least-privilege credentials, input/output validation, prompt injection exposure, privacy, idempotency, retries, escalation, audit logs and rollback.

SENIOR INTERVIEW PRACTICE

Q1. What is the difference between a model generating text and calling a tool?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q2. How do you stop unsafe tool execution?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q3. What does idempotency mean in an automation?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q4. Why should tools return minimal structured data?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

DISCOVERY QUESTIONS

What happens today? What is volume and error cost? Which APIs are mandatory? Which data is sensitive? Which actions need approval? What metric defines a successful pilot?

PROPOSAL ANGLE

Offer a first milestone that reduces the highest uncertainty: API feasibility, retrieval quality, classification accuracy or workflow reliability.

COMMERCIAL NOTE

Estimate from scope/dependencies/risk. Never promise 100% accuracy or full autonomy when acceptable error handling is undefined.

HOMEWORK & PORTFOLIO OUTPUT

- Implement one function-calling loop.
- Document permissions for every tool.
- Create a threat table covering injection, excessive permissions, duplicate execution and malformed arguments.

PORTFOLIO CHECKLIST

- problem statement
- architecture diagram
- setup instructions
- example input/output
- tests or eval cases
- error handling
- security/permissions note
- known limitations
- demo steps

SUBMISSION RUBRIC

Correctness 30% | Reliability 20% | Security 15% | Tests/evals 15% | Documentation 10% | Architecture reasoning 10%.

CLIENT-FACING SUMMARY

Write five sentences: problem, automated flow, where AI is used, how risk is controlled, and expected measurable result. Reuse this in LinkedIn/portfolio/Upwork material.

SELF-ASSESSMENT

Can you justify the architecture, reproduce a failure, diagnose it, and explain scope/success to a client?

NEXT-DAY BRIDGE

Record one unresolved technical question and one business question.

REFERENCE HABIT

Check current official docs for SDK syntax, model/tool availability, limits and platform behavior before production.

ROLE TARGET

n8n / AI Automation Developer

WHY THIS DAY MATTERS

Clients rarely need a model in isolation; they need a dependable business system. Today you will connect the technical topic to architecture, failure handling and a portfolio-ready deliverable.

LEARNING OBJECTIVES

- Understand trigger-action workflow architecture.
- Build deterministic automations around AI steps.
- Use webhooks, schedules, branching and data transformation.
- Decide when AI is necessary and when rules are better.

60-MINUTE DELIVERY PLAN

0-5 min: goals/job relevance; 5-15: core concepts; 15-25: architecture; 25-35: implementation; 35-50: guided lab; 50-56: debugging/security; 56-60: interview check + assignment.

END-OF-DAY EVIDENCE

Leave with implementation notes, a tested artifact, interview answers and a client-facing explanation of tradeoffs.

INSTRUCTOR PROMPTS

- Give one business example where this skill creates measurable value.
- Which part should remain deterministic code?
- What data or permissions make the design risky?
- How would you evaluate the result before release?

PRE-LECTURE PREP

Open the project repository, prepare sandbox credentials/test data, review the previous day and write one question the lab should answer.

5-MINUTE CHECK

Explain today's concept without buzzwords, sketch the architecture from memory, and name the most expensive likely failure.

CORE CONCEPTS - PART I

1. Workflow triggers. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
2. Webhook design. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
3. Nodes and data items. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
4. Expressions and mappings. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
5. IF/Switch branching. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.

TEACHING EMPHASIS

Do not memorize vocabulary. Translate each concept into a design choice and a measurable outcome.

CONCEPT DRILL

Complete: 'I would use this when ____; avoid it when ____; test it by ____.'

COMMON FAILURE PATTERN

Junior implementations often optimize the model call while ignoring validation, permissions, retries, data quality and human exception handling. Those surrounding controls usually determine production success.

SENIOR TAKEAWAY

The more probabilistic the component, the more explicit its measurement and fallback strategy must be.

CORE CONCEPTS - PART II

6. Loops/batching. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
7. Credentials. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
8. HTTP Request node. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
9. AI nodes. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
10. Deterministic vs probabilistic steps. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

SENIOR-LEVEL CONNECTION

Every probabilistic decision needs validation/evaluation; every side effect needs authorization; every integration needs a failure strategy; every quality claim needs evidence.

TRADEOFF EXERCISE

Compare a fast/cheap implementation with a high-reliability implementation. Discuss cost, latency, complexity, accuracy and failure impact.

CLIENT TRANSLATION

Sell the changed business process, not the technical term: what manual step changes, what evidence is used, what becomes faster, and what a human still approves.

REVIEW QUESTION

If this component were wrong for one week, how would the team notice? If the answer is 'they would not', add observability or an eval checkpoint.

ARCHITECTURE WALKTHROUGH

Reference flow:

Trigger -> Normalize -> Validate -> AI step -> Decision branch -> Business-system action -> Notification -> Log/error path

1. Trigger: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
2. Normalize: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
3. Validate: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
4. AI step: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
5. Decision branch: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
6. Business-system action: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
7. Notification: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
8. Log/error path: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.

DESIGN REVIEW QUESTIONS

- Where is untrusted input introduced?
- Where are secrets or sensitive data present?
- Which step is probabilistic?
- Which step changes a system of record?
- What is the human exception path?
- What proves success?

FAILURE-MODE WALKTHROUGH

Test mentally: model timeout, stale data, 429, expired credential, duplicate webhook, unauthorized request. Add a graceful path for relevant failures.

BOUNDARY RULE

Never pass probabilistic output directly into a high-impact side effect. Insert schema validation, business rules, authorization and, where needed, human approval.

DELIVERABLE

Create a technical diagram and a simpler client diagram.

IMPLEMENTATION WALKTHROUGH

Workflow pattern:

Webhook -> Validate payload -> Set normalized fields -> AI classify -> IF confidence >= threshold -> CRM update -> Slack notify
↳ else Human Review Queue

HOW TO READ THIS EXAMPLE

1. Identify component contracts. 2. Separate configuration/secrets. 3. Validate input. 4. Validate model/tool output. 5. Add structured errors and correlation IDs. 6. Prefer explicit schemas over parsing prose.

PRODUCTION EXTENSIONS

Add timeouts, transient retries, rate-limit handling, test fixtures, logging, environment config and a small eval dataset.

TEST PLAN

- happy path
- malformed/missing fields
- timeout / rate-limit
- invalid structured output
- duplicate request
- unauthorized action
- downstream rejection

CODE-REVIEW QUESTIONS

Where can exceptions escape? Which values are trusted? Which calls create side effects? Are secrets loaded safely? Can one request be traced end-to-end? Can prompts/business rules change independently?

OUTSOURCING STANDARD

Deliver code plus config template, setup steps, sample data, tests/evals and known limitations. 'Works on my machine' is not a handoff.

GUIDED LAB - PART I

LAB GOAL

Build an n8n lead-intake workflow: webhook, normalize fields, AI classify, branch by confidence, write to a mock CRM or spreadsheet, and send a notification.

STEP 1 - DEFINE ACCEPTANCE

Write 3-5 observable success conditions before building, including one must-not-happen condition.

STEP 2 - PREPARE TEST DATA

Create normal, edge and malformed examples. Use sandbox/synthetic data.

STEP 3 - BUILD A VERTICAL SLICE

Implement one end-to-end path from trigger/input to validated output. Keep risky side effects mocked or sandboxed first.

STEP 4 - ADD OBSERVABILITY

Record request/workflow ID, timing, decision result, validation result and error category.

STEP 5 - INSPECT INTERMEDIATES

Run manually and inspect each boundary once before adding complexity.

STEP 6 - CAPTURE BASELINE

Record first success rate, latency and estimated cost/task.

STEP 7 - WRITE ACCEPTANCE TESTS

Turn success conditions into repeatable tests/evals, including one case that must escalate.

LAB DELIVERABLE

Commit code/workflow export, sample payloads, architecture image, README and test results.

GUIDED LAB - PART II: HARDENING & DEBUGGING

Test empty input, unexpected type, missing credential, provider timeout, rate limit, low-confidence AI output, invalid structured output, duplicate trigger and downstream API rejection.

DEBUGGING METHOD

1. Reproduce minimally.
2. Locate the failing boundary.
3. Inspect raw input/output safely.
4. Classify: validation, model, network, auth, permission, rate limit, provider, business rule or data.
5. Fix root cause.
6. Add regression test/eval.
7. Document symptom and recovery.

ROOT-CAUSE VS PATCH

Do not add arbitrary delays, broad retries or larger prompts without evidence. Name the root cause and the prevention mechanism.

RECOVERY DESIGN

For each terminal failure choose: retry later, review queue, compensate/rollback, alert operator, or safely discard.

CLIENT-READY FINISH

README sections: Known Limitations and Operations / Recovery.

PORTFOLIO TIP

Include one 'What went wrong' story. A well-explained bug and fix demonstrates seniority better than a perfect demo.

CLIENT SCENARIO

A marketing agency wants every website lead summarized and routed automatically. Define the trigger payload, required fields, AI task, routing logic, error path and expected handoff.

HOW A SENIOR FREELANCER RESPONDS

Restate the business outcome. Ask about process, volume, systems, sensitive data, permissions, exceptions and cost of mistakes. Propose the smallest phase that proves value. Define AI vs deterministic steps, approvals, acceptance criteria and handoff.

SECURITY / RELIABILITY REVIEW

Check least-privilege credentials, input/output validation, prompt injection exposure, privacy, idempotency, retries, escalation, audit logs and rollback.

SENIOR INTERVIEW PRACTICE

Q1. What parts of an n8n workflow should remain deterministic?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q2. How do credentials work conceptually?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q3. How do you make workflows debuggable?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q4. Why is an error branch a first-class design requirement?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

DISCOVERY QUESTIONS

What happens today? What is volume and error cost? Which APIs are mandatory? Which data is sensitive? Which actions need approval? What metric defines a successful pilot?

PROPOSAL ANGLE

Offer a first milestone that reduces the highest uncertainty: API feasibility, retrieval quality, classification accuracy or workflow reliability.

COMMERCIAL NOTE

Estimate from scope/dependencies/risk. Never promise 100% accuracy or full autonomy when acceptable error handling is undefined.

HOMEWORK & PORTFOLIO OUTPUT

- Export your workflow JSON and store it in Git.
- Add a dead-letter/error path.
- Create a one-page workflow diagram for a client README.

PORTFOLIO CHECKLIST

- problem statement
- architecture diagram
- setup instructions
- example input/output
- tests or eval cases
- error handling
- security/permissions note
- known limitations
- demo steps

SUBMISSION RUBRIC

Correctness 30% | Reliability 20% | Security 15% | Tests/evals 15% | Documentation 10% | Architecture reasoning 10%.

CLIENT-FACING SUMMARY

Write five sentences: problem, automated flow, where AI is used, how risk is controlled, and expected measurable result. Reuse this in LinkedIn/portfolio/Upwork material.

SELF-ASSESSMENT

Can you justify the architecture, reproduce a failure, diagnose it, and explain scope/success to a client?

NEXT-DAY BRIDGE

Record one unresolved technical question and one business question.

REFERENCE HABIT

Check current official docs for SDK syntax, model/tool availability, limits and platform behavior before production.

ROLE TARGET

Senior Automation Engineer

WHY THIS DAY MATTERS

Clients rarely need a model in isolation; they need a dependable business system. Today you will connect the technical topic to architecture, failure handling and a portfolio-ready deliverable.

LEARNING OBJECTIVES

- Integrate REST APIs, OAuth/API keys and webhooks reliably.
- Design retries, deduplication and resumable workflows.
- Handle pagination, batching and rate limits.
- Build maintainable n8n projects for clients.

60-MINUTE DELIVERY PLAN

0-5 min: goals/job relevance; 5-15: core concepts; 15-25: architecture; 25-35: implementation; 35-50: guided lab; 50-56: debugging/security; 56-60: interview check + assignment.

END-OF-DAY EVIDENCE

Leave with implementation notes, a tested artifact, interview answers and a client-facing explanation of tradeoffs.

INSTRUCTOR PROMPTS

- Give one business example where this skill creates measurable value.
- Which part should remain deterministic code?
- What data or permissions make the design risky?
- How would you evaluate the result before release?

PRE-LECTURE PREP

Open the project repository, prepare sandbox credentials/test data, review the previous day and write one question the lab should answer.

5-MINUTE CHECK

Explain today's concept without buzzwords, sketch the architecture from memory, and name the most expensive likely failure.

CORE CONCEPTS - PART I

1. HTTP methods and status codes. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
2. REST pagination. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
3. OAuth2/API key patterns. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
4. Webhook signatures. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
5. Retries and backoff. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.

TEACHING EMPHASIS

Do not memorize vocabulary. Translate each concept into a design choice and a measurable outcome.

CONCEPT DRILL

Complete: 'I would use this when ____; avoid it when ____; test it by ____.'

COMMON FAILURE PATTERN

Junior implementations often optimize the model call while ignoring validation, permissions, retries, data quality and human exception handling. Those surrounding controls usually determine production success.

SENIOR TAKEAWAY

The more probabilistic the component, the more explicit its measurement and fallback strategy must be.

CORE CONCEPTS - PART II

6. Rate limiting. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
7. Deduplication keys. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
8. Idempotency. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
9. Sub-workflows. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
10. Environment separation. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

SENIOR-LEVEL CONNECTION

Every probabilistic decision needs validation/evaluation; every side effect needs authorization; every integration needs a failure strategy; every quality claim needs evidence.

TRADEOFF EXERCISE

Compare a fast/cheap implementation with a high-reliability implementation. Discuss cost, latency, complexity, accuracy and failure impact.

CLIENT TRANSLATION

Sell the changed business process, not the technical term: what manual step changes, what evidence is used, what becomes faster, and what a human still approves.

REVIEW QUESTION

If this component were wrong for one week, how would the team notice? If the answer is 'they would not', add observability or an eval checkpoint.

ARCHITECTURE WALKTHROUGH

Reference flow:

Inbound webhook -> Signature check -> Idempotency store -> Business logic -> External API calls -> Retry policy -> Error workflow -> Monitoring/alerts

1. Inbound webhook: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
2. Signature check: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
3. Idempotency store: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
4. Business logic: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
5. External API calls: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
6. Retry policy: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
7. Error workflow: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
8. Monitoring/alerts: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.

DESIGN REVIEW QUESTIONS

- Where is untrusted input introduced?
- Where are secrets or sensitive data present?
- Which step is probabilistic?
- Which step changes a system of record?
- What is the human exception path?
- What proves success?

FAILURE-MODE WALKTHROUGH

Test mentally: model timeout, stale data, 429, expired credential, duplicate webhook, unauthorized request. Add a graceful path for relevant failures.

BOUNDARY RULE

Never pass probabilistic output directly into a high-impact side effect. Insert schema validation, business rules, authorization and, where needed, human approval.

DELIVERABLE

Create a technical diagram and a simpler client diagram.

IMPLEMENTATION WALKTHROUGH

Reliability checklist:

1. Validate source
2. Generate/accept idempotency key
3. Persist processing state
4. Retry only retryable errors
5. Respect provider rate limits
6. Log correlation ID
7. Alert after terminal failure

HOW TO READ THIS EXAMPLE

1. Identify component contracts. 2. Separate configuration/secrets. 3. Validate input. 4. Validate model/tool output. 5. Add structured errors and correlation IDs. 6. Prefer explicit schemas over parsing prose.

PRODUCTION EXTENSIONS

Add timeouts, transient retries, rate-limit handling, test fixtures, logging, environment config and a small eval dataset.

TEST PLAN

- happy path
- malformed/missing fields
- timeout / rate-limit
- invalid structured output
- duplicate request
- unauthorized action
- downstream rejection

CODE-REVIEW QUESTIONS

Where can exceptions escape? Which values are trusted? Which calls create side effects? Are secrets loaded safely? Can one request be traced end-to-end? Can prompts/business rules change independently?

OUTSOURCING STANDARD

Deliver code plus config template, setup steps, sample data, tests/evals and known limitations. 'Works on my machine' is not a handoff.

GUIDED LAB - PART I

LAB GOAL

Extend Day 5 with a real HTTP API. Handle 401, 429, 5xx and invalid JSON separately. Add retry/backoff, a correlation ID and a persistent deduplication check.

STEP 1 - DEFINE ACCEPTANCE

Write 3-5 observable success conditions before building, including one must-not-happen condition.

STEP 2 - PREPARE TEST DATA

Create normal, edge and malformed examples. Use sandbox/synthetic data.

STEP 3 - BUILD A VERTICAL SLICE

Implement one end-to-end path from trigger/input to validated output. Keep risky side effects mocked or sandboxed first.

STEP 4 - ADD OBSERVABILITY

Record request/workflow ID, timing, decision result, validation result and error category.

STEP 5 - INSPECT INTERMEDIATES

Run manually and inspect each boundary once before adding complexity.

STEP 6 - CAPTURE BASELINE

Record first success rate, latency and estimated cost/task.

STEP 7 - WRITE ACCEPTANCE TESTS

Turn success conditions into repeatable tests/evals, including one case that must escalate.

LAB DELIVERABLE

Commit code/workflow export, sample payloads, architecture image, README and test results.

GUIDED LAB - PART II: HARDENING & DEBUGGING

Test empty input, unexpected type, missing credential, provider timeout, rate limit, low-confidence AI output, invalid structured output, duplicate trigger and downstream API rejection.

DEBUGGING METHOD

1. Reproduce minimally.
2. Locate the failing boundary.
3. Inspect raw input/output safely.
4. Classify: validation, model, network, auth, permission, rate limit, provider, business rule or data.
5. Fix root cause.
6. Add regression test/eval.
7. Document symptom and recovery.

ROOT-CAUSE VS PATCH

Do not add arbitrary delays, broad retries or larger prompts without evidence. Name the root cause and the prevention mechanism.

RECOVERY DESIGN

For each terminal failure choose: retry later, review queue, compensate/rollback, alert operator, or safely discard.

CLIENT-READY FINISH

README sections: Known Limitations and Operations / Recovery.

PORTFOLIO TIP

Include one 'What went wrong' story. A well-explained bug and fix demonstrates seniority better than a perfect demo.

CLIENT SCENARIO

A client says, "Sometimes n8n creates duplicate CRM records." Diagnose likely causes and propose an idempotent design rather than simply adding delays.

HOW A SENIOR FREELANCER RESPONDS

Restate the business outcome. Ask about process, volume, systems, sensitive data, permissions, exceptions and cost of mistakes. Propose the smallest phase that proves value. Define AI vs deterministic steps, approvals, acceptance criteria and handoff.

SECURITY / RELIABILITY REVIEW

Check least-privilege credentials, input/output validation, prompt injection exposure, privacy, idempotency, retries, escalation, audit logs and rollback.

SENIOR INTERVIEW PRACTICE

Q1. What failures should be retried?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q2. How do you prevent webhook duplicates?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q3. What is the difference between authentication and authorization?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q4. How do you design for provider rate limits?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

DISCOVERY QUESTIONS

What happens today? What is volume and error cost? Which APIs are mandatory? Which data is sensitive? Which actions need approval? What metric defines a successful pilot?

PROPOSAL ANGLE

Offer a first milestone that reduces the highest uncertainty: API feasibility, retrieval quality, classification accuracy or workflow reliability.

COMMERCIAL NOTE

Estimate from scope/dependencies/risk. Never promise 100% accuracy or full autonomy when acceptable error handling is undefined.

HOMEWORK & PORTFOLIO OUTPUT

- Write a production-readiness checklist for n8n.
- Create a reusable error-handler sub-workflow.
- Document local/dev/prod credential strategy without storing secrets in Git.

PORTFOLIO CHECKLIST

- problem statement
- architecture diagram
- setup instructions
- example input/output
- tests or eval cases
- error handling
- security/permissions note
- known limitations
- demo steps

SUBMISSION RUBRIC

Correctness 30% | Reliability 20% | Security 15% | Tests/evals 15% | Documentation 10% | Architecture reasoning 10%.

CLIENT-FACING SUMMARY

Write five sentences: problem, automated flow, where AI is used, how risk is controlled, and expected measurable result. Reuse this in LinkedIn/portfolio/Upwork material.

SELF-ASSESSMENT

Can you justify the architecture, reproduce a failure, diagnose it, and explain scope/success to a client?

NEXT-DAY BRIDGE

Record one unresolved technical question and one business question.

REFERENCE HABIT

Check current official docs for SDK syntax, model/tool availability, limits and platform behavior before production.

ROLE TARGET

RAG / Knowledge Assistant Developer

WHY THIS DAY MATTERS

Clients rarely need a model in isolation; they need a dependable business system. Today you will connect the technical topic to architecture, failure handling and a portfolio-ready deliverable.

LEARNING OBJECTIVES

- Explain why retrieval-augmented generation is used.
- Create an ingestion and retrieval pipeline.
- Understand embeddings, chunking, similarity and metadata filters.
- Identify common RAG failure modes.

60-MINUTE DELIVERY PLAN

0-5 min: goals/job relevance; 5-15: core concepts; 15-25: architecture; 25-35: implementation; 35-50: guided lab; 50-56: debugging/security; 56-60: interview check + assignment.

END-OF-DAY EVIDENCE

Leave with implementation notes, a tested artifact, interview answers and a client-facing explanation of tradeoffs.

INSTRUCTOR PROMPTS

- Give one business example where this skill creates measurable value.
- Which part should remain deterministic code?
- What data or permissions make the design risky?
- How would you evaluate the result before release?

PRE-LECTURE PREP

Open the project repository, prepare sandbox credentials/test data, review the previous day and write one question the lab should answer.

5-MINUTE CHECK

Explain today's concept without buzzwords, sketch the architecture from memory, and name the most expensive likely failure.

CORE CONCEPTS - PART I

1. RAG architecture. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
2. Embeddings. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
3. Vector similarity. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
4. Chunking. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
5. Overlap. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
6. Metadata. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.

TEACHING EMPHASIS

Do not memorize vocabulary. Translate each concept into a design choice and a measurable outcome.

CONCEPT DRILL

Complete: 'I would use this when ____; avoid it when ____; test it by ____.'

COMMON FAILURE PATTERN

Junior implementations often optimize the model call while ignoring validation, permissions, retries, data quality and human exception handling. Those surrounding controls usually determine production success.

SENIOR TAKEAWAY

The more probabilistic the component, the more explicit its measurement and fallback strategy must be.

CORE CONCEPTS - PART II

7. Vector stores. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
8. Top-k retrieval. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
9. Hybrid search. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
10. Grounding. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
11. Citations. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

SENIOR-LEVEL CONNECTION

Every probabilistic decision needs validation/evaluation; every side effect needs authorization; every integration needs a failure strategy; every quality claim needs evidence.

TRADEOFF EXERCISE

Compare a fast/cheap implementation with a high-reliability implementation. Discuss cost, latency, complexity, accuracy and failure impact.

CLIENT TRANSLATION

Sell the changed business process, not the technical term: what manual step changes, what evidence is used, what becomes faster, and what a human still approves.

REVIEW QUESTION

If this component were wrong for one week, how would the team notice? If the answer is 'they would not', add observability or an eval checkpoint.

ARCHITECTURE WALKTHROUGH

Reference flow:

Documents -> Parse/clean -> Chunk -> Embed -> Vector index -> User query -> Query embedding -> Retrieve -> Rerank/filter -> Prompt with evidence -> Answer + citations

1. Documents: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
2. Parse/clean: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
3. Chunk: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
4. Embed: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
5. Vector index: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
6. User query: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
7. Query embedding: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
8. Retrieve: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
9. Rerank/filter: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
10. Prompt with evidence: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
11. Answer + citations: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.

DESIGN REVIEW QUESTIONS

- Where is untrusted input introduced?
- Where are secrets or sensitive data present?
- Which step is probabilistic?
- Which step changes a system of record?
- What is the human exception path?
- What proves success?

FAILURE-MODE WALKTHROUGH

Test mentally: model timeout, stale data, 429, expired credential, duplicate webhook, unauthorized request. Add a graceful path for relevant failures.

BOUNDARY RULE

Never pass probabilistic output directly into a high-impact side effect. Insert schema validation, business rules, authorization and, where needed, human approval.

DELIVERABLE

Create a technical diagram and a simpler client diagram.

IMPLEMENTATION WALKTHROUGH

```
# RAG pseudocode
chunks = chunk(clean(parse(files)))
vectors = embed(chunks)
index.upsert(vectors, metadata)
retrieved = index.search(embed(question), top_k=6)
answer = llm(question, context=retrieved)
```

HOW TO READ THIS EXAMPLE

1. Identify component contracts. 2. Separate configuration/secrets. 3. Validate input. 4. Validate model/tool output. 5. Add structured errors and correlation IDs. 6. Prefer explicit schemas over parsing prose.

PRODUCTION EXTENSIONS

Add timeouts, transient retries, rate-limit handling, test fixtures, logging, environment config and a small eval dataset.

TEST PLAN

- happy path
- malformed/missing fields
- timeout / rate-limit
- invalid structured output
- duplicate request
- unauthorized action
- downstream rejection

CODE-REVIEW QUESTIONS

Where can exceptions escape? Which values are trusted? Which calls create side effects? Are secrets loaded safely? Can one request be traced end-to-end? Can prompts/business rules change independently?

OUTSOURCING STANDARD

Deliver code plus config template, setup steps, sample data, tests/evals and known limitations. 'Works on my machine' is not a handoff.

GUIDED LAB - PART I

LAB GOAL

Build a small RAG prototype over 10-20 documents. Compare two chunk sizes and measure whether the correct evidence appears in top-k retrieval.

STEP 1 - DEFINE ACCEPTANCE

Write 3-5 observable success conditions before building, including one must-not-happen condition.

STEP 2 - PREPARE TEST DATA

Create normal, edge and malformed examples. Use sandbox/synthetic data.

STEP 3 - BUILD A VERTICAL SLICE

Implement one end-to-end path from trigger/input to validated output. Keep risky side effects mocked or sandboxed first.

STEP 4 - ADD OBSERVABILITY

Record request/workflow ID, timing, decision result, validation result and error category.

STEP 5 - INSPECT INTERMEDIATES

Run manually and inspect each boundary once before adding complexity.

STEP 6 - CAPTURE BASELINE

Record first success rate, latency and estimated cost/task.

STEP 7 - WRITE ACCEPTANCE TESTS

Turn success conditions into repeatable tests/evals, including one case that must escalate.

LAB DELIVERABLE

Commit code/workflow export, sample payloads, architecture image, README and test results.

GUIDED LAB - PART II: HARDENING & DEBUGGING

Test empty input, unexpected type, missing credential, provider timeout, rate limit, low-confidence AI output, invalid structured output, duplicate trigger and downstream API rejection.

DEBUGGING METHOD

1. Reproduce minimally.
2. Locate the failing boundary.
3. Inspect raw input/output safely.
4. Classify: validation, model, network, auth, permission, rate limit, provider, business rule or data.
5. Fix root cause.
6. Add regression test/eval.
7. Document symptom and recovery.

ROOT-CAUSE VS PATCH

Do not add arbitrary delays, broad retries or larger prompts without evidence. Name the root cause and the prevention mechanism.

RECOVERY DESIGN

For each terminal failure choose: retry later, review queue, compensate/rollback, alert operator, or safely discard.

CLIENT-READY FINISH

README sections: Known Limitations and Operations / Recovery.

PORTFOLIO TIP

Include one 'What went wrong' story. A well-explained bug and fix demonstrates seniority better than a perfect demo.

CLIENT SCENARIO

A company wants "ChatGPT trained on our PDFs." Reframe the request: explain RAG vs model training, access control, source freshness, citations and update workflows.

HOW A SENIOR FREELANCER RESPONDS

Restate the business outcome. Ask about process, volume, systems, sensitive data, permissions, exceptions and cost of mistakes. Propose the smallest phase that proves value. Define AI vs deterministic steps, approvals, acceptance criteria and handoff.

SECURITY / RELIABILITY REVIEW

Check least-privilege credentials, input/output validation, prompt injection exposure, privacy, idempotency, retries, escalation, audit logs and rollback.

SENIOR INTERVIEW PRACTICE

Q1. What is an embedding?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q2. Why can poor chunking break RAG?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q3. What is the difference between retrieval quality and generation quality?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q4. When would metadata filtering be essential?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

DISCOVERY QUESTIONS

What happens today? What is volume and error cost? Which APIs are mandatory? Which data is sensitive? Which actions need approval? What metric defines a successful pilot?

PROPOSAL ANGLE

Offer a first milestone that reduces the highest uncertainty: API feasibility, retrieval quality, classification accuracy or workflow reliability.

COMMERCIAL NOTE

Estimate from scope/dependencies/risk. Never promise 100% accuracy or full autonomy when acceptable error handling is undefined.

HOMEWORK & PORTFOLIO OUTPUT

- Create an ingestion script or n8n ingestion workflow.
- Build a 20-question retrieval test set.
- Document at least five RAG failure modes and how you would diagnose each.

PORTFOLIO CHECKLIST

- problem statement
- architecture diagram
- setup instructions
- example input/output
- tests or eval cases
- error handling
- security/permissions note
- known limitations
- demo steps

SUBMISSION RUBRIC

Correctness 30% | Reliability 20% | Security 15% | Tests/evals 15% | Documentation 10% | Architecture reasoning 10%.

CLIENT-FACING SUMMARY

Write five sentences: problem, automated flow, where AI is used, how risk is controlled, and expected measurable result. Reuse this in LinkedIn/portfolio/Upwork material.

SELF-ASSESSMENT

Can you justify the architecture, reproduce a failure, diagnose it, and explain scope/success to a client?

NEXT-DAY BRIDGE

Record one unresolved technical question and one business question.

REFERENCE HABIT

Check current official docs for SDK syntax, model/tool availability, limits and platform behavior before production.

ROLE TARGET

Senior RAG Engineer

WHY THIS DAY MATTERS

Clients rarely need a model in isolation; they need a dependable business system. Today you will connect the technical topic to architecture, failure handling and a portfolio-ready deliverable.

LEARNING OBJECTIVES

- Turn a prototype RAG system into a maintainable service.
- Design source sync, metadata, permissions and deletion.
- Evaluate retrieval and answer quality separately.
- Implement citation and abstention behavior.

60-MINUTE DELIVERY PLAN

0-5 min: goals/job relevance; 5-15: core concepts; 15-25: architecture; 25-35: implementation; 35-50: guided lab; 50-56: debugging/security; 56-60: interview check + assignment.

END-OF-DAY EVIDENCE

Leave with implementation notes, a tested artifact, interview answers and a client-facing explanation of tradeoffs.

INSTRUCTOR PROMPTS

- Give one business example where this skill creates measurable value.
- Which part should remain deterministic code?
- What data or permissions make the design risky?
- How would you evaluate the result before release?

PRE-LECTURE PREP

Open the project repository, prepare sandbox credentials/test data, review the previous day and write one question the lab should answer.

5-MINUTE CHECK

Explain today's concept without buzzwords, sketch the architecture from memory, and name the most expensive likely failure.

CORE CONCEPTS - PART I

1. Document lifecycle. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
2. Incremental indexing. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
3. Content hashing. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
4. Source-of-truth sync. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
5. Permission-aware retrieval. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.

TEACHING EMPHASIS

Do not memorize vocabulary. Translate each concept into a design choice and a measurable outcome.

CONCEPT DRILL

Complete: 'I would use this when ____; avoid it when ____; test it by ____.'

COMMON FAILURE PATTERN

Junior implementations often optimize the model call while ignoring validation, permissions, retries, data quality and human exception handling. Those surrounding controls usually determine production success.

SENIOR TAKEAWAY

The more probabilistic the component, the more explicit its measurement and fallback strategy must be.

CORE CONCEPTS - PART II

6. Reranking. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
7. Query rewriting. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
8. Citation verification. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
9. Abstention. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
10. RAG evaluation. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

SENIOR-LEVEL CONNECTION

Every probabilistic decision needs validation/evaluation; every side effect needs authorization; every integration needs a failure strategy; every quality claim needs evidence.

TRADEOFF EXERCISE

Compare a fast/cheap implementation with a high-reliability implementation. Discuss cost, latency, complexity, accuracy and failure impact.

CLIENT TRANSLATION

Sell the changed business process, not the technical term: what manual step changes, what evidence is used, what becomes faster, and what a human still approves.

REVIEW QUESTION

If this component were wrong for one week, how would the team notice? If the answer is 'they would not', add observability or an eval checkpoint.

ARCHITECTURE WALKTHROUGH

Reference flow:

Source connectors -> Sync scheduler -> Parser -> Chunk/version store -> Embedding pipeline -> Permission metadata -> Retriever -> Reranker -> Answer generator -> Citation validator -> Eval harness

1. Source connectors: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
2. Sync scheduler: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
3. Parser: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
4. Chunk/version store: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
5. Embedding pipeline: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
6. Permission metadata: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
7. Retriever: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
8. Reranker: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
9. Answer generator: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
10. Citation validator: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
11. Eval harness: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.

DESIGN REVIEW QUESTIONS

- Where is untrusted input introduced?
- Where are secrets or sensitive data present?
- Which step is probabilistic?
- Which step changes a system of record?
- What is the human exception path?
- What proves success?

FAILURE-MODE WALKTHROUGH

Test mentally: model timeout, stale data, 429, expired credential, duplicate webhook, unauthorized request. Add a graceful path for relevant failures.

BOUNDARY RULE

Never pass probabilistic output directly into a high-impact side effect. Insert schema validation, business rules, authorization and, where needed, human approval.

DELIVERABLE

Create a technical diagram and a simpler client diagram.

IMPLEMENTATION WALKTHROUGH

Evaluation dimensions:

- retrieval_recall: did relevant evidence appear?
- groundedness: are claims supported?
- citation_accuracy: do cited chunks support claims?
- abstention: does system refuse when evidence is absent?
- latency/cost: within client SLA?

HOW TO READ THIS EXAMPLE

1. Identify component contracts. 2. Separate configuration/secrets. 3. Validate input. 4. Validate model/tool output. 5. Add structured errors and correlation IDs. 6. Prefer explicit schemas over parsing prose.

PRODUCTION EXTENSIONS

Add timeouts, transient retries, rate-limit handling, test fixtures, logging, environment config and a small eval dataset.

TEST PLAN

- happy path
- malformed/missing fields
- timeout / rate-limit
- invalid structured output
- duplicate request
- unauthorized action
- downstream rejection

CODE-REVIEW QUESTIONS

Where can exceptions escape? Which values are trusted? Which calls create side effects? Are secrets loaded safely? Can one request be traced end-to-end? Can prompts/business rules change independently?

OUTSOURCING STANDARD

Deliver code plus config template, setup steps, sample data, tests/evals and known limitations. 'Works on my machine' is not a handoff.

GUIDED LAB - PART I

LAB GOAL

Add citations and an "I do not know from the provided knowledge base" policy. Create 30 eval questions: answerable, partially answerable and unanswerable.

STEP 1 - DEFINE ACCEPTANCE

Write 3-5 observable success conditions before building, including one must-not-happen condition.

STEP 2 - PREPARE TEST DATA

Create normal, edge and malformed examples. Use sandbox/synthetic data.

STEP 3 - BUILD A VERTICAL SLICE

Implement one end-to-end path from trigger/input to validated output. Keep risky side effects mocked or sandboxed first.

STEP 4 - ADD OBSERVABILITY

Record request/workflow ID, timing, decision result, validation result and error category.

STEP 5 - INSPECT INTERMEDIATES

Run manually and inspect each boundary once before adding complexity.

STEP 6 - CAPTURE BASELINE

Record first success rate, latency and estimated cost/task.

STEP 7 - WRITE ACCEPTANCE TESTS

Turn success conditions into repeatable tests/evals, including one case that must escalate.

LAB DELIVERABLE

Commit code/workflow export, sample payloads, architecture image, README and test results.

GUIDED LAB - PART II: HARDENING & DEBUGGING

Test empty input, unexpected type, missing credential, provider timeout, rate limit, low-confidence AI output, invalid structured output, duplicate trigger and downstream API rejection.

DEBUGGING METHOD

1. Reproduce minimally.
2. Locate the failing boundary.
3. Inspect raw input/output safely.
4. Classify: validation, model, network, auth, permission, rate limit, provider, business rule or data.
5. Fix root cause.
6. Add regression test/eval.
7. Document symptom and recovery.

ROOT-CAUSE VS PATCH

Do not add arbitrary delays, broad retries or larger prompts without evidence. Name the root cause and the prevention mechanism.

RECOVERY DESIGN

For each terminal failure choose: retry later, review queue, compensate/rollback, alert operator, or safely discard.

CLIENT-READY FINISH

README sections: Known Limitations and Operations / Recovery.

PORTFOLIO TIP

Include one 'What went wrong' story. A well-explained bug and fix demonstrates seniority better than a perfect demo.

CLIENT SCENARIO

A HR team needs a policy assistant where users may only retrieve policies for their region. Design metadata and permission filtering so access control happens before context reaches the model.

HOW A SENIOR FREELANCER RESPONDS

Restate the business outcome. Ask about process, volume, systems, sensitive data, permissions, exceptions and cost of mistakes. Propose the smallest phase that proves value. Define AI vs deterministic steps, approvals, acceptance criteria and handoff.

SECURITY / RELIABILITY REVIEW

Check least-privilege credentials, input/output validation, prompt injection exposure, privacy, idempotency, retries, escalation, audit logs and rollback.

SENIOR INTERVIEW PRACTICE

Q1. How do you keep a RAG index fresh?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q2. How do you evaluate retrieval separately from generation?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q3. Why is source deletion important?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q4. How do you implement permission-aware RAG?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

DISCOVERY QUESTIONS

What happens today? What is volume and error cost? Which APIs are mandatory? Which data is sensitive? Which actions need approval? What metric defines a successful pilot?

PROPOSAL ANGLE

Offer a first milestone that reduces the highest uncertainty: API feasibility, retrieval quality, classification accuracy or workflow reliability.

COMMERCIAL NOTE

Estimate from scope/dependencies/risk. Never promise 100% accuracy or full autonomy when acceptable error handling is undefined.

HOMEWORK & PORTFOLIO OUTPUT

- Add source IDs, timestamps and document versions.
- Create an eval report with pass/fail examples.
- Write a production migration plan from prototype vector store to managed storage.

PORTFOLIO CHECKLIST

- problem statement
- architecture diagram
- setup instructions
- example input/output
- tests or eval cases
- error handling
- security/permissions note
- known limitations
- demo steps

SUBMISSION RUBRIC

Correctness 30% | Reliability 20% | Security 15% | Tests/evals 15% | Documentation 10% | Architecture reasoning 10%.

CLIENT-FACING SUMMARY

Write five sentences: problem, automated flow, where AI is used, how risk is controlled, and expected measurable result. Reuse this in LinkedIn/portfolio/Upwork material.

SELF-ASSESSMENT

Can you justify the architecture, reproduce a failure, diagnose it, and explain scope/success to a client?

NEXT-DAY BRIDGE

Record one unresolved technical question and one business question.

REFERENCE HABIT

Check current official docs for SDK syntax, model/tool availability, limits and platform behavior before production.

ROLE TARGET

AI Agent Developer

WHY THIS DAY MATTERS

Clients rarely need a model in isolation; they need a dependable business system. Today you will connect the technical topic to architecture, failure handling and a portfolio-ready deliverable.

LEARNING OBJECTIVES

- Explain the difference between workflows and agents.
- Design bounded agents with tools, memory and clear stop conditions.
- Recognize where agents create unnecessary risk or cost.
- Create human-in-the-loop checkpoints.

60-MINUTE DELIVERY PLAN

0-5 min: goals/job relevance; 5-15: core concepts; 15-25: architecture; 25-35: implementation; 35-50: guided lab; 50-56: debugging/security; 56-60: interview check + assignment.

END-OF-DAY EVIDENCE

Leave with implementation notes, a tested artifact, interview answers and a client-facing explanation of tradeoffs.

INSTRUCTOR PROMPTS

- Give one business example where this skill creates measurable value.
- Which part should remain deterministic code?
- What data or permissions make the design risky?
- How would you evaluate the result before release?

PRE-LECTURE PREP

Open the project repository, prepare sandbox credentials/test data, review the previous day and write one question the lab should answer.

5-MINUTE CHECK

Explain today's concept without buzzwords, sketch the architecture from memory, and name the most expensive likely failure.

CORE CONCEPTS - PART I

1. Agent loop. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
2. Planning vs direct response. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
3. Tool use. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
4. Short-term memory. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
5. Long-term memory. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.

TEACHING EMPHASIS

Do not memorize vocabulary. Translate each concept into a design choice and a measurable outcome.

CONCEPT DRILL

Complete: 'I would use this when ____; avoid it when ____; test it by ____.'

COMMON FAILURE PATTERN

Junior implementations often optimize the model call while ignoring validation, permissions, retries, data quality and human exception handling. Those surrounding controls usually determine production success.

SENIOR TAKEAWAY

The more probabilistic the component, the more explicit its measurement and fallback strategy must be.

CORE CONCEPTS - PART II

6. State. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
7. Stop conditions. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
8. Guardrails. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
9. Human approval. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
10. Workflow vs agent. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

SENIOR-LEVEL CONNECTION

Every probabilistic decision needs validation/evaluation; every side effect needs authorization; every integration needs a failure strategy; every quality claim needs evidence.

TRADEOFF EXERCISE

Compare a fast/cheap implementation with a high-reliability implementation. Discuss cost, latency, complexity, accuracy and failure impact.

CLIENT TRANSLATION

Sell the changed business process, not the technical term: what manual step changes, what evidence is used, what becomes faster, and what a human still approves.

REVIEW QUESTION

If this component were wrong for one week, how would the team notice? If the answer is 'they would not', add observability or an eval checkpoint.

ARCHITECTURE WALKTHROUGH

Reference flow:

Goal -> Agent state -> Model reasoning -> Tool choice -> Tool execution -> State update -> Policy/guardrail -> Repeat or finish -> Human approval

1. Goal: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
2. Agent state: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
3. Model reasoning: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
4. Tool choice: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
5. Tool execution: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
6. State update: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
7. Policy/guardrail: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
8. Repeat or finish: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
9. Human approval: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.

DESIGN REVIEW QUESTIONS

- Where is untrusted input introduced?
- Where are secrets or sensitive data present?
- Which step is probabilistic?
- Which step changes a system of record?
- What is the human exception path?
- What proves success?

FAILURE-MODE WALKTHROUGH

Test mentally: model timeout, stale data, 429, expired credential, duplicate webhook, unauthorized request. Add a graceful path for relevant failures.

BOUNDARY RULE

Never pass probabilistic output directly into a high-impact side effect. Insert schema validation, business rules, authorization and, where needed, human approval.

DELIVERABLE

Create a technical diagram and a simpler client diagram.

IMPLEMENTATION WALKTHROUGH

Agent design rule:

Use deterministic code for known sequence + rules.

Use an agent when the next step legitimately depends on context and tool results.

Bound the agent with: allowed tools, max turns, budgets, approvals, stop conditions.

HOW TO READ THIS EXAMPLE

1. Identify component contracts. 2. Separate configuration/secrets. 3. Validate input. 4. Validate model/tool output. 5. Add structured errors and correlation IDs. 6. Prefer explicit schemas over parsing prose.

PRODUCTION EXTENSIONS

Add timeouts, transient retries, rate-limit handling, test fixtures, logging, environment config and a small eval dataset.

TEST PLAN

- happy path
- malformed/missing fields
- timeout / rate-limit
- invalid structured output
- duplicate request
- unauthorized action
- downstream rejection

CODE-REVIEW QUESTIONS

Where can exceptions escape? Which values are trusted? Which calls create side effects? Are secrets loaded safely? Can one request be traced end-to-end? Can prompts/business rules change independently?

OUTSOURCING STANDARD

Deliver code plus config template, setup steps, sample data, tests/evals and known limitations. 'Works on my machine' is not a handoff.

GUIDED LAB - PART I

LAB GOAL

Design an agent that researches a customer account using only read-only tools, drafts a briefing, and asks a human before any outbound action.

STEP 1 - DEFINE ACCEPTANCE

Write 3-5 observable success conditions before building, including one must-not-happen condition.

STEP 2 - PREPARE TEST DATA

Create normal, edge and malformed examples. Use sandbox/synthetic data.

STEP 3 - BUILD A VERTICAL SLICE

Implement one end-to-end path from trigger/input to validated output. Keep risky side effects mocked or sandboxed first.

STEP 4 - ADD OBSERVABILITY

Record request/workflow ID, timing, decision result, validation result and error category.

STEP 5 - INSPECT INTERMEDIATES

Run manually and inspect each boundary once before adding complexity.

STEP 6 - CAPTURE BASELINE

Record first success rate, latency and estimated cost/task.

STEP 7 - WRITE ACCEPTANCE TESTS

Turn success conditions into repeatable tests/evals, including one case that must escalate.

LAB DELIVERABLE

Commit code/workflow export, sample payloads, architecture image, README and test results.

GUIDED LAB - PART II: HARDENING & DEBUGGING

Test empty input, unexpected type, missing credential, provider timeout, rate limit, low-confidence AI output, invalid structured output, duplicate trigger and downstream API rejection.

DEBUGGING METHOD

1. Reproduce minimally.
2. Locate the failing boundary.
3. Inspect raw input/output safely.
4. Classify: validation, model, network, auth, permission, rate limit, provider, business rule or data.
5. Fix root cause.
6. Add regression test/eval.
7. Document symptom and recovery.

ROOT-CAUSE VS PATCH

Do not add arbitrary delays, broad retries or larger prompts without evidence. Name the root cause and the prevention mechanism.

RECOVERY DESIGN

For each terminal failure choose: retry later, review queue, compensate/rollback, alert operator, or safely discard.

CLIENT-READY FINISH

README sections: Known Limitations and Operations / Recovery.

PORTFOLIO TIP

Include one 'What went wrong' story. A well-explained bug and fix demonstrates seniority better than a perfect demo.

CLIENT SCENARIO

A founder asks for "a fully autonomous sales agent." Convert this into a safer staged system with research automation, draft generation, review, and measurable constraints.

HOW A SENIOR FREELANCER RESPONDS

Restate the business outcome. Ask about process, volume, systems, sensitive data, permissions, exceptions and cost of mistakes. Propose the smallest phase that proves value. Define AI vs deterministic steps, approvals, acceptance criteria and handoff.

SECURITY / RELIABILITY REVIEW

Check least-privilege credentials, input/output validation, prompt injection exposure, privacy, idempotency, retries, escalation, audit logs and rollback.

SENIOR INTERVIEW PRACTICE

Q1. What makes an agent different from a workflow?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q2. What kinds of memory exist?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q3. How do you keep an agent from looping forever?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q4. Where should human approval be inserted?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

DISCOVERY QUESTIONS

What happens today? What is volume and error cost? Which APIs are mandatory? Which data is sensitive? Which actions need approval? What metric defines a successful pilot?

PROPOSAL ANGLE

Offer a first milestone that reduces the highest uncertainty: API feasibility, retrieval quality, classification accuracy or workflow reliability.

COMMERCIAL NOTE

Estimate from scope/dependencies/risk. Never promise 100% accuracy or full autonomy when acceptable error handling is undefined.

HOMEWORK & PORTFOLIO OUTPUT

- Write an agent specification: goal, tools, state, permissions, max turns and success criteria.
- Create a tool-risk matrix.
- Implement a simple bounded loop in code or n8n.

PORTFOLIO CHECKLIST

- problem statement
- architecture diagram
- setup instructions
- example input/output
- tests or eval cases
- error handling
- security/permissions note
- known limitations
- demo steps

SUBMISSION RUBRIC

Correctness 30% | Reliability 20% | Security 15% | Tests/evals 15% | Documentation 10% | Architecture reasoning 10%.

CLIENT-FACING SUMMARY

Write five sentences: problem, automated flow, where AI is used, how risk is controlled, and expected measurable result. Reuse this in LinkedIn/portfolio/Upwork material.

SELF-ASSESSMENT

Can you justify the architecture, reproduce a failure, diagnose it, and explain scope/success to a client?

NEXT-DAY BRIDGE

Record one unresolved technical question and one business question.

REFERENCE HABIT

Check current official docs for SDK syntax, model/tool availability, limits and platform behavior before production.

ROLE TARGET

OpenAI Agent Application Engineer

WHY THIS DAY MATTERS

Clients rarely need a model in isolation; they need a dependable business system. Today you will connect the technical topic to architecture, failure handling and a portfolio-ready deliverable.

LEARNING OBJECTIVES

- Build an agent using the current Agents SDK primitives.
- Use tools, handoffs, sessions and tracing appropriately.
- Understand server-managed model/tool interaction vs app-managed state.
- Design multi-agent systems only when specialization adds value.

60-MINUTE DELIVERY PLAN

0-5 min: goals/job relevance; 5-15: core concepts; 15-25: architecture; 25-35: implementation; 35-50: guided lab; 50-56: debugging/security; 56-60: interview check + assignment.

END-OF-DAY EVIDENCE

Leave with implementation notes, a tested artifact, interview answers and a client-facing explanation of tradeoffs.

INSTRUCTOR PROMPTS

- Give one business example where this skill creates measurable value.
- Which part should remain deterministic code?
- What data or permissions make the design risky?
- How would you evaluate the result before release?

PRE-LECTURE PREP

Open the project repository, prepare sandbox credentials/test data, review the previous day and write one question the lab should answer.

5-MINUTE CHECK

Explain today's concept without buzzwords, sketch the architecture from memory, and name the most expensive likely failure.

CORE CONCEPTS - PART I

1. Agent and Runner. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
2. Responses-based orchestration. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
3. Tools. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
4. Handoffs. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
5. Sessions. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.

TEACHING EMPHASIS

Do not memorize vocabulary. Translate each concept into a design choice and a measurable outcome.

CONCEPT DRILL

Complete: 'I would use this when ____; avoid it when ____; test it by ____.'

COMMON FAILURE PATTERN

Junior implementations often optimize the model call while ignoring validation, permissions, retries, data quality and human exception handling. Those surrounding controls usually determine production success.

SENIOR TAKEAWAY

The more probabilistic the component, the more explicit its measurement and fallback strategy must be.

CORE CONCEPTS - PART II

6. Tracing. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
7. Usage tracking. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
8. Guardrails. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
9. Human-in-the-loop. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
10. Multi-agent delegation. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

SENIOR-LEVEL CONNECTION

Every probabilistic decision needs validation/evaluation; every side effect needs authorization; every integration needs a failure strategy; every quality claim needs evidence.

TRADEOFF EXERCISE

Compare a fast/cheap implementation with a high-reliability implementation. Discuss cost, latency, complexity, accuracy and failure impact.

CLIENT TRANSLATION

Sell the changed business process, not the technical term: what manual step changes, what evidence is used, what becomes faster, and what a human still approves.

REVIEW QUESTION

If this component were wrong for one week, how would the team notice? If the answer is 'they would not', add observability or an eval checkpoint.

ARCHITECTURE WALKTHROUGH

Reference flow:

User -> Triage agent -> Specialist handoff -> Tools -> Session memory -> Trace/evaluation -> Final response

1. User: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
2. Triage agent: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
3. Specialist handoff: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
4. Tools: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
5. Session memory: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
6. Trace/evaluation: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
7. Final response: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.

DESIGN REVIEW QUESTIONS

- Where is untrusted input introduced?
- Where are secrets or sensitive data present?
- Which step is probabilistic?
- Which step changes a system of record?
- What is the human exception path?
- What proves success?

FAILURE-MODE WALKTHROUGH

Test mentally: model timeout, stale data, 429, expired credential, duplicate webhook, unauthorized request. Add a graceful path for relevant failures.

BOUNDARY RULE

Never pass probabilistic output directly into a high-impact side effect. Insert schema validation, business rules, authorization and, where needed, human approval.

DELIVERABLE

Create a technical diagram and a simpler client diagram.

IMPLEMENTATION WALKTHROUGH

```
from agents import Agent, Runner
triage = Agent(
    name="Triage",
    instructions="Route requests to the right specialist.",
    handoffs=[support_agent, sales_agent],
)
result = Runner.run_sync(triage, user_input)
print(result.final_output)
```

HOW TO READ THIS EXAMPLE

1. Identify component contracts. 2. Separate configuration/secrets. 3. Validate input. 4. Validate model/tool output. 5. Add structured errors and correlation IDs. 6. Prefer explicit schemas over parsing prose.

PRODUCTION EXTENSIONS

Add timeouts, transient retries, rate-limit handling, test fixtures, logging, environment config and a small eval dataset.

TEST PLAN

- happy path
- malformed/missing fields
- timeout / rate-limit
- invalid structured output
- duplicate request
- unauthorized action
- downstream rejection

CODE-REVIEW QUESTIONS

Where can exceptions escape? Which values are trusted? Which calls create side effects? Are secrets loaded safely? Can one request be traced end-to-end? Can prompts/business rules change independently?

OUTSOURCING STANDARD

Deliver code plus config template, setup steps, sample data, tests/evals and known limitations. 'Works on my machine' is not a handoff.

GUIDED LAB - PART I

LAB GOAL

Build a triage agent with two specialists and at least one read-only tool. Record traces and test ten routing examples including ambiguous requests.

STEP 1 - DEFINE ACCEPTANCE

Write 3-5 observable success conditions before building, including one must-not-happen condition.

STEP 2 - PREPARE TEST DATA

Create normal, edge and malformed examples. Use sandbox/synthetic data.

STEP 3 - BUILD A VERTICAL SLICE

Implement one end-to-end path from trigger/input to validated output. Keep risky side effects mocked or sandboxed first.

STEP 4 - ADD OBSERVABILITY

Record request/workflow ID, timing, decision result, validation result and error category.

STEP 5 - INSPECT INTERMEDIATES

Run manually and inspect each boundary once before adding complexity.

STEP 6 - CAPTURE BASELINE

Record first success rate, latency and estimated cost/task.

STEP 7 - WRITE ACCEPTANCE TESTS

Turn success conditions into repeatable tests/evals, including one case that must escalate.

LAB DELIVERABLE

Commit code/workflow export, sample payloads, architecture image, README and test results.

GUIDED LAB - PART II: HARDENING & DEBUGGING

Test empty input, unexpected type, missing credential, provider timeout, rate limit, low-confidence AI output, invalid structured output, duplicate trigger and downstream API rejection.

DEBUGGING METHOD

1. Reproduce minimally.
2. Locate the failing boundary.
3. Inspect raw input/output safely.
4. Classify: validation, model, network, auth, permission, rate limit, provider, business rule or data.
5. Fix root cause.
6. Add regression test/eval.
7. Document symptom and recovery.

ROOT-CAUSE VS PATCH

Do not add arbitrary delays, broad retries or larger prompts without evidence. Name the root cause and the prevention mechanism.

RECOVERY DESIGN

For each terminal failure choose: retry later, review queue, compensate/rollback, alert operator, or safely discard.

CLIENT-READY FINISH

README sections: Known Limitations and Operations / Recovery.

PORTFOLIO TIP

Include one 'What went wrong' story. A well-explained bug and fix demonstrates seniority better than a perfect demo.

CLIENT SCENARIO

A SaaS company wants one "super agent" for support, billing and sales. Compare one-agent-with-tools vs triage-plus-specialists and justify the simpler architecture where possible.

HOW A SENIOR FREELANCER RESPONDS

Restate the business outcome. Ask about process, volume, systems, sensitive data, permissions, exceptions and cost of mistakes. Propose the smallest phase that proves value. Define AI vs deterministic steps, approvals, acceptance criteria and handoff.

SECURITY / RELIABILITY REVIEW

Check least-privilege credentials, input/output validation, prompt injection exposure, privacy, idempotency, retries, escalation, audit logs and rollback.

SENIOR INTERVIEW PRACTICE

Q1. When should you use handoffs?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q2. What is session memory for?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q3. Why are traces valuable?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q4. How do you monitor token usage and agent cost?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

DISCOVERY QUESTIONS

What happens today? What is volume and error cost? Which APIs are mandatory? Which data is sensitive? Which actions need approval? What metric defines a successful pilot?

PROPOSAL ANGLE

Offer a first milestone that reduces the highest uncertainty: API feasibility, retrieval quality, classification accuracy or workflow reliability.

COMMERCIAL NOTE

Estimate from scope/dependencies/risk. Never promise 100% accuracy or full autonomy when acceptable error handling is undefined.

HOMEWORK & PORTFOLIO OUTPUT

- Create a multi-agent diagram and README.
- Add a human approval step for a sensitive tool.
- Create routing eval cases and report accuracy.

PORTFOLIO CHECKLIST

- problem statement
- architecture diagram
- setup instructions
- example input/output
- tests or eval cases
- error handling
- security/permissions note
- known limitations
- demo steps

SUBMISSION RUBRIC

Correctness 30% | Reliability 20% | Security 15% | Tests/evals 15% | Documentation 10% | Architecture reasoning 10%.

CLIENT-FACING SUMMARY

Write five sentences: problem, automated flow, where AI is used, how risk is controlled, and expected measurable result. Reuse this in LinkedIn/portfolio/Upwork material.

SELF-ASSESSMENT

Can you justify the architecture, reproduce a failure, diagnose it, and explain scope/success to a client?

NEXT-DAY BRIDGE

Record one unresolved technical question and one business question.

REFERENCE HABIT

Check current official docs for SDK syntax, model/tool availability, limits and platform behavior before production.

ROLE TARGET

Agent Orchestration Engineer

WHY THIS DAY MATTERS

Clients rarely need a model in isolation; they need a dependable business system. Today you will connect the technical topic to architecture, failure handling and a portfolio-ready deliverable.

LEARNING OBJECTIVES

- Model agent systems as state, nodes and edges.
- Use deterministic routing around agentic steps.
- Design resumable and human-review workflows.
- Know when LangGraph-level control is justified.

60-MINUTE DELIVERY PLAN

0-5 min: goals/job relevance; 5-15: core concepts; 15-25: architecture; 25-35: implementation; 35-50: guided lab; 50-56: debugging/security; 56-60: interview check + assignment.

END-OF-DAY EVIDENCE

Leave with implementation notes, a tested artifact, interview answers and a client-facing explanation of tradeoffs.

INSTRUCTOR PROMPTS

- Give one business example where this skill creates measurable value.
- Which part should remain deterministic code?
- What data or permissions make the design risky?
- How would you evaluate the result before release?

PRE-LECTURE PREP

Open the project repository, prepare sandbox credentials/test data, review the previous day and write one question the lab should answer.

5-MINUTE CHECK

Explain today's concept without buzzwords, sketch the architecture from memory, and name the most expensive likely failure.

CORE CONCEPTS - PART I

1. State. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
2. Nodes. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
3. Edges. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
4. Conditional routing. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
5. Checkpoints. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.

TEACHING EMPHASIS

Do not memorize vocabulary. Translate each concept into a design choice and a measurable outcome.

CONCEPT DRILL

Complete: 'I would use this when ____; avoid it when ____; test it by ____.'

COMMON FAILURE PATTERN

Junior implementations often optimize the model call while ignoring validation, permissions, retries, data quality and human exception handling. Those surrounding controls usually determine production success.

SENIOR TAKEAWAY

The more probabilistic the component, the more explicit its measurement and fallback strategy must be.

CORE CONCEPTS - PART II

6. Persistence. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
7. Human-in-the-loop. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
8. Cycles. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
9. Subgraphs. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
10. Long-running workflows. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

SENIOR-LEVEL CONNECTION

Every probabilistic decision needs validation/evaluation; every side effect needs authorization; every integration needs a failure strategy; every quality claim needs evidence.

TRADEOFF EXERCISE

Compare a fast/cheap implementation with a high-reliability implementation. Discuss cost, latency, complexity, accuracy and failure impact.

CLIENT TRANSLATION

Sell the changed business process, not the technical term: what manual step changes, what evidence is used, what becomes faster, and what a human still approves.

REVIEW QUESTION

If this component were wrong for one week, how would the team notice? If the answer is 'they would not', add observability or an eval checkpoint.

ARCHITECTURE WALKTHROUGH

Reference flow:

Input state -> Classifier node -> Retrieve node -> Reason node -> Approval node -> Action node -> Checkpoint store -> End

1. Input state: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
2. Classifier node: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
3. Retrieve node: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
4. Reason node: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
5. Approval node: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
6. Action node: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
7. Checkpoint store: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
8. End: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.

DESIGN REVIEW QUESTIONS

- Where is untrusted input introduced?
- Where are secrets or sensitive data present?
- Which step is probabilistic?
- Which step changes a system of record?
- What is the human exception path?
- What proves success?

FAILURE-MODE WALKTHROUGH

Test mentally: model timeout, stale data, 429, expired credential, duplicate webhook, unauthorized request. Add a graceful path for relevant failures.

BOUNDARY RULE

Never pass probabilistic output directly into a high-impact side effect. Insert schema validation, business rules, authorization and, where needed, human approval.

DELIVERABLE

Create a technical diagram and a simpler client diagram.

IMPLEMENTATION WALKTHROUGH

Graph mental model
State = {messages, customer_id, evidence, decision, approval}
Nodes = {classify, retrieve, draft, approve, execute}
Edges = deterministic transitions + conditional branches
Persist checkpoints before risky or long-running steps.

HOW TO READ THIS EXAMPLE

1. Identify component contracts. 2. Separate configuration/secrets. 3. Validate input. 4. Validate model/tool output. 5. Add structured errors and correlation IDs. 6. Prefer explicit schemas over parsing prose.

PRODUCTION EXTENSIONS

Add timeouts, transient retries, rate-limit handling, test fixtures, logging, environment config and a small eval dataset.

TEST PLAN

- happy path
- malformed/missing fields
- timeout / rate-limit
- invalid structured output
- duplicate request
- unauthorized action
- downstream rejection

CODE-REVIEW QUESTIONS

Where can exceptions escape? Which values are trusted? Which calls create side effects? Are secrets loaded safely? Can one request be traced end-to-end? Can prompts/business rules change independently?

OUTSOURCING STANDARD

Deliver code plus config template, setup steps, sample data, tests/evals and known limitations. 'Works on my machine' is not a handoff.

GUIDED LAB - PART I

LAB GOAL

Design a support escalation graph with classify -> retrieve -> draft -> confidence branch -> human approval -> ticket update. Persist state so the process can resume.

STEP 1 - DEFINE ACCEPTANCE

Write 3-5 observable success conditions before building, including one must-not-happen condition.

STEP 2 - PREPARE TEST DATA

Create normal, edge and malformed examples. Use sandbox/synthetic data.

STEP 3 - BUILD A VERTICAL SLICE

Implement one end-to-end path from trigger/input to validated output. Keep risky side effects mocked or sandboxed first.

STEP 4 - ADD OBSERVABILITY

Record request/workflow ID, timing, decision result, validation result and error category.

STEP 5 - INSPECT INTERMEDIATES

Run manually and inspect each boundary once before adding complexity.

STEP 6 - CAPTURE BASELINE

Record first success rate, latency and estimated cost/task.

STEP 7 - WRITE ACCEPTANCE TESTS

Turn success conditions into repeatable tests/evals, including one case that must escalate.

LAB DELIVERABLE

Commit code/workflow export, sample payloads, architecture image, README and test results.

GUIDED LAB - PART II: HARDENING & DEBUGGING

Test empty input, unexpected type, missing credential, provider timeout, rate limit, low-confidence AI output, invalid structured output, duplicate trigger and downstream API rejection.

DEBUGGING METHOD

1. Reproduce minimally.
2. Locate the failing boundary.
3. Inspect raw input/output safely.
4. Classify: validation, model, network, auth, permission, rate limit, provider, business rule or data.
5. Fix root cause.
6. Add regression test/eval.
7. Document symptom and recovery.

ROOT-CAUSE VS PATCH

Do not add arbitrary delays, broad retries or larger prompts without evidence. Name the root cause and the prevention mechanism.

RECOVERY DESIGN

For each terminal failure choose: retry later, review queue, compensate/rollback, alert operator, or safely discard.

CLIENT-READY FINISH

README sections: Known Limitations and Operations / Recovery.

PORTFOLIO TIP

Include one 'What went wrong' story. A well-explained bug and fix demonstrates seniority better than a perfect demo.

CLIENT SCENARIO

A finance workflow may pause for hours awaiting approval. Explain why resumable state and deterministic edges can be a better fit than a single conversational agent loop.

HOW A SENIOR FREELANCER RESPONDS

Restate the business outcome. Ask about process, volume, systems, sensitive data, permissions, exceptions and cost of mistakes. Propose the smallest phase that proves value. Define AI vs deterministic steps, approvals, acceptance criteria and handoff.

SECURITY / RELIABILITY REVIEW

Check least-privilege credentials, input/output validation, prompt injection exposure, privacy, idempotency, retries, escalation, audit logs and rollback.

SENIOR INTERVIEW PRACTICE

Q1. What are state, nodes and edges?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q2. Why is persistence important?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q3. When is a graph preferable to a simple agent?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q4. How do checkpoints help with human review?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

DISCOVERY QUESTIONS

What happens today? What is volume and error cost? Which APIs are mandatory? Which data is sensitive? Which actions need approval? What metric defines a successful pilot?

PROPOSAL ANGLE

Offer a first milestone that reduces the highest uncertainty: API feasibility, retrieval quality, classification accuracy or workflow reliability.

COMMERCIAL NOTE

Estimate from scope/dependencies/risk. Never promise 100% accuracy or full autonomy when acceptable error handling is undefined.

HOMEWORK & PORTFOLIO OUTPUT

- Implement or diagram a six-node graph.
- Define the state schema explicitly.
- List which edges are deterministic and which are model-influenced.

PORTFOLIO CHECKLIST

- problem statement
- architecture diagram
- setup instructions
- example input/output
- tests or eval cases
- error handling
- security/permissions note
- known limitations
- demo steps

SUBMISSION RUBRIC

Correctness 30% | Reliability 20% | Security 15% | Tests/evals 15% | Documentation 10% | Architecture reasoning 10%.

CLIENT-FACING SUMMARY

Write five sentences: problem, automated flow, where AI is used, how risk is controlled, and expected measurable result. Reuse this in LinkedIn/portfolio/Upwork material.

SELF-ASSESSMENT

Can you justify the architecture, reproduce a failure, diagnose it, and explain scope/success to a client?

NEXT-DAY BRIDGE

Record one unresolved technical question and one business question.

REFERENCE HABIT

Check current official docs for SDK syntax, model/tool availability, limits and platform behavior before production.

ROLE TARGET

AI Automation Architect

WHY THIS DAY MATTERS

Clients rarely need a model in isolation; they need a dependable business system. Today you will connect the technical topic to architecture, failure handling and a portfolio-ready deliverable.

LEARNING OBJECTIVES

- Translate business processes into automation opportunities.
- Choose appropriate combinations of rules, LLMs, RAG and agents.
- Design measurable operational outcomes.
- Avoid automating unstable or undefined processes.

60-MINUTE DELIVERY PLAN

0-5 min: goals/job relevance; 5-15: core concepts; 15-25: architecture; 25-35: implementation; 35-50: guided lab; 50-56: debugging/security; 56-60: interview check + assignment.

END-OF-DAY EVIDENCE

Leave with implementation notes, a tested artifact, interview answers and a client-facing explanation of tradeoffs.

INSTRUCTOR PROMPTS

- Give one business example where this skill creates measurable value.
- Which part should remain deterministic code?
- What data or permissions make the design risky?
- How would you evaluate the result before release?

PRE-LECTURE PREP

Open the project repository, prepare sandbox credentials/test data, review the previous day and write one question the lab should answer.

5-MINUTE CHECK

Explain today's concept without buzzwords, sketch the architecture from memory, and name the most expensive likely failure.

CORE CONCEPTS - PART I

1. Process discovery. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
2. SOP mapping. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
3. Trigger/action decomposition. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
4. Classification. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
5. Extraction. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
6. Drafting. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.

TEACHING EMPHASIS

Do not memorize vocabulary. Translate each concept into a design choice and a measurable outcome.

CONCEPT DRILL

Complete: 'I would use this when ____; avoid it when ____; test it by ____.'

COMMON FAILURE PATTERN

Junior implementations often optimize the model call while ignoring validation, permissions, retries, data quality and human exception handling. Those surrounding controls usually determine production success.

SENIOR TAKEAWAY

The more probabilistic the component, the more explicit its measurement and fallback strategy must be.

CORE CONCEPTS - PART II

7. Routing. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

8. Approval. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

9. CRM integration. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

10. Support automation. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

11. Sales operations. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

12. Back-office workflows. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

SENIOR-LEVEL CONNECTION

Every probabilistic decision needs validation/evaluation; every side effect needs authorization; every integration needs a failure strategy; every quality claim needs evidence.

TRADEOFF EXERCISE

Compare a fast/cheap implementation with a high-reliability implementation. Discuss cost, latency, complexity, accuracy and failure impact.

CLIENT TRANSLATION

Sell the changed business process, not the technical term: what manual step changes, what evidence is used, what becomes faster, and what a human still approves.

REVIEW QUESTION

If this component were wrong for one week, how would the team notice? If the answer is 'they would not', add observability or an eval checkpoint.

ARCHITECTURE WALKTHROUGH

Reference flow:

Business event -> Data normalization -> AI interpretation -> Policy/rules -> System-of-record update -> Human exception queue -> Metrics dashboard

1. Business event: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
2. Data normalization: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
3. AI interpretation: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
4. Policy/rules: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
5. System-of-record update: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
6. Human exception queue: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
7. Metrics dashboard: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.

DESIGN REVIEW QUESTIONS

- Where is untrusted input introduced?
- Where are secrets or sensitive data present?
- Which step is probabilistic?
- Which step changes a system of record?
- What is the human exception path?
- What proves success?

FAILURE-MODE WALKTHROUGH

Test mentally: model timeout, stale data, 429, expired credential, duplicate webhook, unauthorized request. Add a graceful path for relevant failures.

BOUNDARY RULE

Never pass probabilistic output directly into a high-impact side effect. Insert schema validation, business rules, authorization and, where needed, human approval.

DELIVERABLE

Create a technical diagram and a simpler client diagram.

IMPLEMENTATION WALKTHROUGH

Automation opportunity score:

Value = volume * minutes_saved * labor_cost

Risk = error_impact * uncertainty * permission_level

Start with high-value / low-risk processes and add autonomy only after evidence.

HOW TO READ THIS EXAMPLE

1. Identify component contracts. 2. Separate configuration/secrets. 3. Validate input. 4. Validate model/tool output. 5. Add structured errors and correlation IDs. 6. Prefer explicit schemas over parsing prose.

PRODUCTION EXTENSIONS

Add timeouts, transient retries, rate-limit handling, test fixtures, logging, environment config and a small eval dataset.

TEST PLAN

- happy path
- malformed/missing fields
- timeout / rate-limit
- invalid structured output
- duplicate request
- unauthorized action
- downstream rejection

CODE-REVIEW QUESTIONS

Where can exceptions escape? Which values are trusted? Which calls create side effects? Are secrets loaded safely? Can one request be traced end-to-end? Can prompts/business rules change independently?

OUTSOURCING STANDARD

Deliver code plus config template, setup steps, sample data, tests/evals and known limitations. 'Works on my machine' is not a handoff.

GUIDED LAB - PART I

LAB GOAL

Map three workflows for an e-commerce company: support triage, lead qualification and weekly operations reporting. For each, mark deterministic, AI and human-review steps.

STEP 1 - DEFINE ACCEPTANCE

Write 3-5 observable success conditions before building, including one must-not-happen condition.

STEP 2 - PREPARE TEST DATA

Create normal, edge and malformed examples. Use sandbox/synthetic data.

STEP 3 - BUILD A VERTICAL SLICE

Implement one end-to-end path from trigger/input to validated output. Keep risky side effects mocked or sandboxed first.

STEP 4 - ADD OBSERVABILITY

Record request/workflow ID, timing, decision result, validation result and error category.

STEP 5 - INSPECT INTERMEDIATES

Run manually and inspect each boundary once before adding complexity.

STEP 6 - CAPTURE BASELINE

Record first success rate, latency and estimated cost/task.

STEP 7 - WRITE ACCEPTANCE TESTS

Turn success conditions into repeatable tests/evals, including one case that must escalate.

LAB DELIVERABLE

Commit code/workflow export, sample payloads, architecture image, README and test results.

GUIDED LAB - PART II: HARDENING & DEBUGGING

Test empty input, unexpected type, missing credential, provider timeout, rate limit, low-confidence AI output, invalid structured output, duplicate trigger and downstream API rejection.

DEBUGGING METHOD

1. Reproduce minimally.
2. Locate the failing boundary.
3. Inspect raw input/output safely.
4. Classify: validation, model, network, auth, permission, rate limit, provider, business rule or data.
5. Fix root cause.
6. Add regression test/eval.
7. Document symptom and recovery.

ROOT-CAUSE VS PATCH

Do not add arbitrary delays, broad retries or larger prompts without evidence. Name the root cause and the prevention mechanism.

RECOVERY DESIGN

For each terminal failure choose: retry later, review queue, compensate/rollback, alert operator, or safely discard.

CLIENT-READY FINISH

README sections: Known Limitations and Operations / Recovery.

PORTFOLIO TIP

Include one 'What went wrong' story. A well-explained bug and fix demonstrates seniority better than a perfect demo.

CLIENT SCENARIO

A business owner lists 20 manual tasks. Prioritize the first automation using volume, value, technical complexity, data readiness and risk—not novelty.

HOW A SENIOR FREELANCER RESPONDS

Restate the business outcome. Ask about process, volume, systems, sensitive data, permissions, exceptions and cost of mistakes. Propose the smallest phase that proves value. Define AI vs deterministic steps, approvals, acceptance criteria and handoff.

SECURITY / RELIABILITY REVIEW

Check least-privilege credentials, input/output validation, prompt injection exposure, privacy, idempotency, retries, escalation, audit logs and rollback.

SENIOR INTERVIEW PRACTICE

Q1. How do you decide what to automate first?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q2. What makes a process automation-ready?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q3. How do you quantify ROI?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q4. What should remain a human exception path?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

DISCOVERY QUESTIONS

What happens today? What is volume and error cost? Which APIs are mandatory? Which data is sensitive? Which actions need approval? What metric defines a successful pilot?

PROPOSAL ANGLE

Offer a first milestone that reduces the highest uncertainty: API feasibility, retrieval quality, classification accuracy or workflow reliability.

COMMERCIAL NOTE

Estimate from scope/dependencies/risk. Never promise 100% accuracy or full autonomy when acceptable error handling is undefined.

HOMEWORK & PORTFOLIO OUTPUT

- Create an automation discovery worksheet.
- Estimate monthly savings for three candidate workflows.
- Write a phased roadmap: proof of concept, pilot, production, optimization.

PORTFOLIO CHECKLIST

- problem statement
- architecture diagram
- setup instructions
- example input/output
- tests or eval cases
- error handling
- security/permissions note
- known limitations
- demo steps

SUBMISSION RUBRIC

Correctness 30% | Reliability 20% | Security 15% | Tests/evals 15% | Documentation 10% | Architecture reasoning 10%.

CLIENT-FACING SUMMARY

Write five sentences: problem, automated flow, where AI is used, how risk is controlled, and expected measurable result. Reuse this in LinkedIn/portfolio/Upwork material.

SELF-ASSESSMENT

Can you justify the architecture, reproduce a failure, diagnose it, and explain scope/success to a client?

NEXT-DAY BRIDGE

Record one unresolved technical question and one business question.

REFERENCE HABIT

Check current official docs for SDK syntax, model/tool availability, limits and platform behavior before production.

ROLE TARGET

AI Document Automation Engineer

WHY THIS DAY MATTERS

Clients rarely need a model in isolation; they need a dependable business system. Today you will connect the technical topic to architecture, failure handling and a portfolio-ready deliverable.

LEARNING OBJECTIVES

- Build reliable pipelines for messy business documents.
- Use AI for extraction while enforcing deterministic validation.
- Handle PDFs, spreadsheets, forms and email attachments.
- Design confidence-based exception handling.

60-MINUTE DELIVERY PLAN

0-5 min: goals/job relevance; 5-15: core concepts; 15-25: architecture; 25-35: implementation; 35-50: guided lab; 50-56: debugging/security; 56-60: interview check + assignment.

END-OF-DAY EVIDENCE

Leave with implementation notes, a tested artifact, interview answers and a client-facing explanation of tradeoffs.

INSTRUCTOR PROMPTS

- Give one business example where this skill creates measurable value.
- Which part should remain deterministic code?
- What data or permissions make the design risky?
- How would you evaluate the result before release?

PRE-LECTURE PREP

Open the project repository, prepare sandbox credentials/test data, review the previous day and write one question the lab should answer.

5-MINUTE CHECK

Explain today's concept without buzzwords, sketch the architecture from memory, and name the most expensive likely failure.

CORE CONCEPTS - PART I

1. Document ingestion. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
2. Text extraction/OCR boundary. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
3. Structured extraction. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
4. Schema validation. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
5. Normalization. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.

TEACHING EMPHASIS

Do not memorize vocabulary. Translate each concept into a design choice and a measurable outcome.

CONCEPT DRILL

Complete: 'I would use this when ____; avoid it when ____; test it by ____.'

COMMON FAILURE PATTERN

Junior implementations often optimize the model call while ignoring validation, permissions, retries, data quality and human exception handling. Those surrounding controls usually determine production success.

SENIOR TAKEAWAY

The more probabilistic the component, the more explicit its measurement and fallback strategy must be.

CORE CONCEPTS - PART II

6. Cross-field validation. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
7. Confidence. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
8. Duplicate detection. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
9. Human review. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
10. Data lineage. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

SENIOR-LEVEL CONNECTION

Every probabilistic decision needs validation/evaluation; every side effect needs authorization; every integration needs a failure strategy; every quality claim needs evidence.

TRADEOFF EXERCISE

Compare a fast/cheap implementation with a high-reliability implementation. Discuss cost, latency, complexity, accuracy and failure impact.

CLIENT TRANSLATION

Sell the changed business process, not the technical term: what manual step changes, what evidence is used, what becomes faster, and what a human still approves.

REVIEW QUESTION

If this component were wrong for one week, how would the team notice? If the answer is 'they would not', add observability or an eval checkpoint.

ARCHITECTURE WALKTHROUGH

Reference flow:

Email/upload -> File scanner -> Text extraction -> Document classifier -> Field extractor -> Schema validation -> Business-rule validation -> Database/ERP -> Exception queue

1. Email/upload: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
2. File scanner: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
3. Text extraction: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
4. Document classifier: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
5. Field extractor: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
6. Schema validation: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
7. Business-rule validation: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
8. Database/ERP: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
9. Exception queue: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.

DESIGN REVIEW QUESTIONS

- Where is untrusted input introduced?
- Where are secrets or sensitive data present?
- Which step is probabilistic?
- Which step changes a system of record?
- What is the human exception path?
- What proves success?

FAILURE-MODE WALKTHROUGH

Test mentally: model timeout, stale data, 429, expired credential, duplicate webhook, unauthorized request. Add a graceful path for relevant failures.

BOUNDARY RULE

Never pass probabilistic output directly into a high-impact side effect. Insert schema validation, business rules, authorization and, where needed, human approval.

DELIVERABLE

Create a technical diagram and a simpler client diagram.

IMPLEMENTATION WALKTHROUGH

Extraction rule:

LLM output is a candidate record, not truth.

Validate types, required fields, totals, dates, currency, identifiers and cross-field consistency before writing to the system of record.

HOW TO READ THIS EXAMPLE

1. Identify component contracts. 2. Separate configuration/secrets. 3. Validate input. 4. Validate model/tool output. 5. Add structured errors and correlation IDs. 6. Prefer explicit schemas over parsing prose.

PRODUCTION EXTENSIONS

Add timeouts, transient retries, rate-limit handling, test fixtures, logging, environment config and a small eval dataset.

TEST PLAN

- happy path
- malformed/missing fields
- timeout / rate-limit
- invalid structured output
- duplicate request
- unauthorized action
- downstream rejection

CODE-REVIEW QUESTIONS

Where can exceptions escape? Which values are trusted? Which calls create side effects? Are secrets loaded safely? Can one request be traced end-to-end? Can prompts/business rules change independently?

OUTSOURCING STANDARD

Deliver code plus config template, setup steps, sample data, tests/evals and known limitations. 'Works on my machine' is not a handoff.

GUIDED LAB - PART I

LAB GOAL

Create an invoice or purchase-order extraction workflow using synthetic/sample documents. Validate required fields and route low-confidence or inconsistent records to review.

STEP 1 - DEFINE ACCEPTANCE

Write 3-5 observable success conditions before building, including one must-not-happen condition.

STEP 2 - PREPARE TEST DATA

Create normal, edge and malformed examples. Use sandbox/synthetic data.

STEP 3 - BUILD A VERTICAL SLICE

Implement one end-to-end path from trigger/input to validated output. Keep risky side effects mocked or sandboxed first.

STEP 4 - ADD OBSERVABILITY

Record request/workflow ID, timing, decision result, validation result and error category.

STEP 5 - INSPECT INTERMEDIATES

Run manually and inspect each boundary once before adding complexity.

STEP 6 - CAPTURE BASELINE

Record first success rate, latency and estimated cost/task.

STEP 7 - WRITE ACCEPTANCE TESTS

Turn success conditions into repeatable tests/evals, including one case that must escalate.

LAB DELIVERABLE

Commit code/workflow export, sample payloads, architecture image, README and test results.

GUIDED LAB - PART II: HARDENING & DEBUGGING

Test empty input, unexpected type, missing credential, provider timeout, rate limit, low-confidence AI output, invalid structured output, duplicate trigger and downstream API rejection.

DEBUGGING METHOD

1. Reproduce minimally.
2. Locate the failing boundary.
3. Inspect raw input/output safely.
4. Classify: validation, model, network, auth, permission, rate limit, provider, business rule or data.
5. Fix root cause.
6. Add regression test/eval.
7. Document symptom and recovery.

ROOT-CAUSE VS PATCH

Do not add arbitrary delays, broad retries or larger prompts without evidence. Name the root cause and the prevention mechanism.

RECOVERY DESIGN

For each terminal failure choose: retry later, review queue, compensate/rollback, alert operator, or safely discard.

CLIENT-READY FINISH

README sections: Known Limitations and Operations / Recovery.

PORTFOLIO TIP

Include one 'What went wrong' story. A well-explained bug and fix demonstrates seniority better than a perfect demo.

CLIENT SCENARIO

An accounting firm wants invoices entered automatically. Define acceptable error rates, fields, validation rules, duplicate detection and a review queue before automating ERP writes.

HOW A SENIOR FREELANCER RESPONDS

Restate the business outcome. Ask about process, volume, systems, sensitive data, permissions, exceptions and cost of mistakes. Propose the smallest phase that proves value. Define AI vs deterministic steps, approvals, acceptance criteria and handoff.

SECURITY / RELIABILITY REVIEW

Check least-privilege credentials, input/output validation, prompt injection exposure, privacy, idempotency, retries, escalation, audit logs and rollback.

SENIOR INTERVIEW PRACTICE

Q1. Why is schema validation insufficient by itself?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q2. How do you detect duplicate documents?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q3. What is data lineage?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q4. Where should confidence thresholds be used?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

DISCOVERY QUESTIONS

What happens today? What is volume and error cost? Which APIs are mandatory? Which data is sensitive? Which actions need approval? What metric defines a successful pilot?

PROPOSAL ANGLE

Offer a first milestone that reduces the highest uncertainty: API feasibility, retrieval quality, classification accuracy or workflow reliability.

COMMERCIAL NOTE

Estimate from scope/dependencies/risk. Never promise 100% accuracy or full autonomy when acceptable error handling is undefined.

HOMEWORK & PORTFOLIO OUTPUT

- Create a JSON schema for an invoice.
- Write ten cross-field validation rules.
- Produce a before/after process diagram and ROI estimate.

PORTFOLIO CHECKLIST

- problem statement
- architecture diagram
- setup instructions
- example input/output
- tests or eval cases
- error handling
- security/permissions note
- known limitations
- demo steps

SUBMISSION RUBRIC

Correctness 30% | Reliability 20% | Security 15% | Tests/evals 15% | Documentation 10% | Architecture reasoning 10%.

CLIENT-FACING SUMMARY

Write five sentences: problem, automated flow, where AI is used, how risk is controlled, and expected measurable result. Reuse this in LinkedIn/portfolio/Upwork material.

SELF-ASSESSMENT

Can you justify the architecture, reproduce a failure, diagnose it, and explain scope/success to a client?

NEXT-DAY BRIDGE

Record one unresolved technical question and one business question.

REFERENCE HABIT

Check current official docs for SDK syntax, model/tool availability, limits and platform behavior before production.

ROLE TARGET

Multimodal / Voice AI Developer

WHY THIS DAY MATTERS

Clients rarely need a model in isolation; they need a dependable business system. Today you will connect the technical topic to architecture, failure handling and a portfolio-ready deliverable.

LEARNING OBJECTIVES

- Design systems that accept text, images and audio.
- Understand speech/voice workflow components and latency constraints.
- Create safe realtime agent boundaries.
- Identify high-value multimodal outsourcing use cases.

60-MINUTE DELIVERY PLAN

0-5 min: goals/job relevance; 5-15: core concepts; 15-25: architecture; 25-35: implementation; 35-50: guided lab; 50-56: debugging/security; 56-60: interview check + assignment.

END-OF-DAY EVIDENCE

Leave with implementation notes, a tested artifact, interview answers and a client-facing explanation of tradeoffs.

INSTRUCTOR PROMPTS

- Give one business example where this skill creates measurable value.
- Which part should remain deterministic code?
- What data or permissions make the design risky?
- How would you evaluate the result before release?

PRE-LECTURE PREP

Open the project repository, prepare sandbox credentials/test data, review the previous day and write one question the lab should answer.

5-MINUTE CHECK

Explain today's concept without buzzwords, sketch the architecture from memory, and name the most expensive likely failure.

CORE CONCEPTS - PART I

1. Vision inputs. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
2. Audio transcription. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
3. Text-to-speech. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
4. Realtime interaction. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
5. Streaming. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.

TEACHING EMPHASIS

Do not memorize vocabulary. Translate each concept into a design choice and a measurable outcome.

CONCEPT DRILL

Complete: 'I would use this when ____; avoid it when ____; test it by ____.'

COMMON FAILURE PATTERN

Junior implementations often optimize the model call while ignoring validation, permissions, retries, data quality and human exception handling. Those surrounding controls usually determine production success.

SENIOR TAKEAWAY

The more probabilistic the component, the more explicit its measurement and fallback strategy must be.

CORE CONCEPTS - PART II

6. Turn-taking. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
7. Latency budget. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
8. Tool calls in voice flows. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
9. Telephony integration. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
10. Escalation. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

SENIOR-LEVEL CONNECTION

Every probabilistic decision needs validation/evaluation; every side effect needs authorization; every integration needs a failure strategy; every quality claim needs evidence.

TRADEOFF EXERCISE

Compare a fast/cheap implementation with a high-reliability implementation. Discuss cost, latency, complexity, accuracy and failure impact.

CLIENT TRANSLATION

Sell the changed business process, not the technical term: what manual step changes, what evidence is used, what becomes faster, and what a human still approves.

REVIEW QUESTION

If this component were wrong for one week, how would the team notice? If the answer is 'they would not', add observability or an eval checkpoint.

ARCHITECTURE WALKTHROUGH

Reference flow:

Call/image input -> Media transport -> Model/realtime session -> Tool layer -> Business system -> Human transfer -> Transcript/logs

1. Call/image input: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
2. Media transport: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
3. Model/realtime session: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
4. Tool layer: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
5. Business system: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
6. Human transfer: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
7. Transcript/logs: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.

DESIGN REVIEW QUESTIONS

- Where is untrusted input introduced?
- Where are secrets or sensitive data present?
- Which step is probabilistic?
- Which step changes a system of record?
- What is the human exception path?
- What proves success?

FAILURE-MODE WALKTHROUGH

Test mentally: model timeout, stale data, 429, expired credential, duplicate webhook, unauthorized request. Add a graceful path for relevant failures.

BOUNDARY RULE

Never pass probabilistic output directly into a high-impact side effect. Insert schema validation, business rules, authorization and, where needed, human approval.

DELIVERABLE

Create a technical diagram and a simpler client diagram.

IMPLEMENTATION WALKTHROUGH

Realtime latency budget example:

network + audio buffering + model response + tool latency + synthesis

If a tool takes seconds, acknowledge progress or transfer to asynchronous workflow instead of creating dead air.

HOW TO READ THIS EXAMPLE

1. Identify component contracts. 2. Separate configuration/secrets. 3. Validate input. 4. Validate model/tool output. 5. Add structured errors and correlation IDs. 6. Prefer explicit schemas over parsing prose.

PRODUCTION EXTENSIONS

Add timeouts, transient retries, rate-limit handling, test fixtures, logging, environment config and a small eval dataset.

TEST PLAN

- happy path
- malformed/missing fields
- timeout / rate-limit
- invalid structured output
- duplicate request
- unauthorized action
- downstream rejection

CODE-REVIEW QUESTIONS

Where can exceptions escape? Which values are trusted? Which calls create side effects? Are secrets loaded safely? Can one request be traced end-to-end? Can prompts/business rules change independently?

OUTSOURCING STANDARD

Deliver code plus config template, setup steps, sample data, tests/evals and known limitations. 'Works on my machine' is not a handoff.

GUIDED LAB - PART I

LAB GOAL

Design a voice receptionist or visual-inspection assistant. Include user identity checks, permitted actions, escalation, transcript handling and failure recovery.

STEP 1 - DEFINE ACCEPTANCE

Write 3-5 observable success conditions before building, including one must-not-happen condition.

STEP 2 - PREPARE TEST DATA

Create normal, edge and malformed examples. Use sandbox/synthetic data.

STEP 3 - BUILD A VERTICAL SLICE

Implement one end-to-end path from trigger/input to validated output. Keep risky side effects mocked or sandboxed first.

STEP 4 - ADD OBSERVABILITY

Record request/workflow ID, timing, decision result, validation result and error category.

STEP 5 - INSPECT INTERMEDIATES

Run manually and inspect each boundary once before adding complexity.

STEP 6 - CAPTURE BASELINE

Record first success rate, latency and estimated cost/task.

STEP 7 - WRITE ACCEPTANCE TESTS

Turn success conditions into repeatable tests/evals, including one case that must escalate.

LAB DELIVERABLE

Commit code/workflow export, sample payloads, architecture image, README and test results.

GUIDED LAB - PART II: HARDENING & DEBUGGING

Test empty input, unexpected type, missing credential, provider timeout, rate limit, low-confidence AI output, invalid structured output, duplicate trigger and downstream API rejection.

DEBUGGING METHOD

1. Reproduce minimally.
2. Locate the failing boundary.
3. Inspect raw input/output safely.
4. Classify: validation, model, network, auth, permission, rate limit, provider, business rule or data.
5. Fix root cause.
6. Add regression test/eval.
7. Document symptom and recovery.

ROOT-CAUSE VS PATCH

Do not add arbitrary delays, broad retries or larger prompts without evidence. Name the root cause and the prevention mechanism.

RECOVERY DESIGN

For each terminal failure choose: retry later, review queue, compensate/rollback, alert operator, or safely discard.

CLIENT-READY FINISH

README sections: Known Limitations and Operations / Recovery.

PORTFOLIO TIP

Include one 'What went wrong' story. A well-explained bug and fix demonstrates seniority better than a perfect demo.

CLIENT SCENARIO

A clinic wants an AI phone receptionist. Define safe scope: scheduling and FAQs may be automated; medical advice and sensitive decisions require strict boundaries and human escalation.

HOW A SENIOR FREELANCER RESPONDS

Restate the business outcome. Ask about process, volume, systems, sensitive data, permissions, exceptions and cost of mistakes. Propose the smallest phase that proves value. Define AI vs deterministic steps, approvals, acceptance criteria and handoff.

SECURITY / RELIABILITY REVIEW

Check least-privilege credentials, input/output validation, prompt injection exposure, privacy, idempotency, retries, escalation, audit logs and rollback.

SENIOR INTERVIEW PRACTICE

Q1. What makes voice agents harder than chat?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q2. What contributes to latency?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q3. How do you handle sensitive actions in realtime?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q4. What data should be retained from calls?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

DISCOVERY QUESTIONS

What happens today? What is volume and error cost? Which APIs are mandatory? Which data is sensitive? Which actions need approval? What metric defines a successful pilot?

PROPOSAL ANGLE

Offer a first milestone that reduces the highest uncertainty: API feasibility, retrieval quality, classification accuracy or workflow reliability.

COMMERCIAL NOTE

Estimate from scope/dependencies/risk. Never promise 100% accuracy or full autonomy when acceptable error handling is undefined.

HOMEWORK & PORTFOLIO OUTPUT

- Create a sequence diagram for a voice agent.
- Define latency targets and fallbacks.
- Write a privacy and retention checklist for audio/transcripts.

PORTFOLIO CHECKLIST

- problem statement
- architecture diagram
- setup instructions
- example input/output
- tests or eval cases
- error handling
- security/permissions note
- known limitations
- demo steps

SUBMISSION RUBRIC

Correctness 30% | Reliability 20% | Security 15% | Tests/evals 15% | Documentation 10% | Architecture reasoning 10%.

CLIENT-FACING SUMMARY

Write five sentences: problem, automated flow, where AI is used, how risk is controlled, and expected measurable result. Reuse this in LinkedIn/portfolio/Upwork material.

SELF-ASSESSMENT

Can you justify the architecture, reproduce a failure, diagnose it, and explain scope/success to a client?

NEXT-DAY BRIDGE

Record one unresolved technical question and one business question.

REFERENCE HABIT

Check current official docs for SDK syntax, model/tool availability, limits and platform behavior before production.

ROLE TARGET

AI Reliability / Security Engineer

WHY THIS DAY MATTERS

Clients rarely need a model in isolation; they need a dependable business system. Today you will connect the technical topic to architecture, failure handling and a portfolio-ready deliverable.

LEARNING OBJECTIVES

- Treat AI quality as an engineering discipline.
- Build evaluation datasets and graders around business outcomes.
- Design guardrails for inputs, outputs and tools.
- Understand core AI security and privacy risks.

60-MINUTE DELIVERY PLAN

0-5 min: goals/job relevance; 5-15: core concepts; 15-25: architecture; 25-35: implementation; 35-50: guided lab; 50-56: debugging/security; 56-60: interview check + assignment.

END-OF-DAY EVIDENCE

Leave with implementation notes, a tested artifact, interview answers and a client-facing explanation of tradeoffs.

INSTRUCTOR PROMPTS

- Give one business example where this skill creates measurable value.
- Which part should remain deterministic code?
- What data or permissions make the design risky?
- How would you evaluate the result before release?

PRE-LECTURE PREP

Open the project repository, prepare sandbox credentials/test data, review the previous day and write one question the lab should answer.

5-MINUTE CHECK

Explain today's concept without buzzwords, sketch the architecture from memory, and name the most expensive likely failure.

CORE CONCEPTS - PART I

1. Evaluation datasets. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
2. Golden examples. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
3. Regression tests. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
4. Groundedness. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
5. Task success. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
6. Tool-call correctness. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.

TEACHING EMPHASIS

Do not memorize vocabulary. Translate each concept into a design choice and a measurable outcome.

CONCEPT DRILL

Complete: 'I would use this when ____; avoid it when ____; test it by ____.'

COMMON FAILURE PATTERN

Junior implementations often optimize the model call while ignoring validation, permissions, retries, data quality and human exception handling. Those surrounding controls usually determine production success.

SENIOR TAKEAWAY

The more probabilistic the component, the more explicit its measurement and fallback strategy must be.

CORE CONCEPTS - PART II

7. Prompt injection. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
8. Data leakage. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
9. Secrets. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
10. PII minimization. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
11. Guardrails. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
12. Red teaming. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

SENIOR-LEVEL CONNECTION

Every probabilistic decision needs validation/evaluation; every side effect needs authorization; every integration needs a failure strategy; every quality claim needs evidence.

TRADEOFF EXERCISE

Compare a fast/cheap implementation with a high-reliability implementation. Discuss cost, latency, complexity, accuracy and failure impact.

CLIENT TRANSLATION

Sell the changed business process, not the technical term: what manual step changes, what evidence is used, what becomes faster, and what a human still approves.

REVIEW QUESTION

If this component were wrong for one week, how would the team notice? If the answer is 'they would not', add observability or an eval checkpoint.

ARCHITECTURE WALKTHROUGH

Reference flow:

Test dataset -> System under test -> Trace capture -> Graders/metrics -> Failure analysis -> Prompt/tool changes -> Regression gate

1. Test dataset: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
2. System under test: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
3. Trace capture: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
4. Graders/metrics: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
5. Failure analysis: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
6. Prompt/tool changes: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
7. Regression gate: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.

DESIGN REVIEW QUESTIONS

- Where is untrusted input introduced?
- Where are secrets or sensitive data present?
- Which step is probabilistic?
- Which step changes a system of record?
- What is the human exception path?
- What proves success?

FAILURE-MODE WALKTHROUGH

Test mentally: model timeout, stale data, 429, expired credential, duplicate webhook, unauthorized request. Add a graceful path for relevant failures.

BOUNDARY RULE

Never pass probabilistic output directly into a high-impact side effect. Insert schema validation, business rules, authorization and, where needed, human approval.

DELIVERABLE

Create a technical diagram and a simpler client diagram.

IMPLEMENTATION WALKTHROUGH

Release gate example:

- task_success >= target
 - critical_tool_errors == 0
 - citation_accuracy >= target
 - prompt_injection suite passes
 - cost/latency within budget
- No AI change ships only because "the demo looked good."

HOW TO READ THIS EXAMPLE

1. Identify component contracts. 2. Separate configuration/secrets. 3. Validate input. 4. Validate model/tool output. 5. Add structured errors and correlation IDs. 6. Prefer explicit schemas over parsing prose.

PRODUCTION EXTENSIONS

Add timeouts, transient retries, rate-limit handling, test fixtures, logging, environment config and a small eval dataset.

TEST PLAN

- happy path
- malformed/missing fields
- timeout / rate-limit
- invalid structured output
- duplicate request
- unauthorized action
- downstream rejection

CODE-REVIEW QUESTIONS

Where can exceptions escape? Which values are trusted? Which calls create side effects? Are secrets loaded safely? Can one request be traced end-to-end? Can prompts/business rules change independently?

OUTSOURCING STANDARD

Deliver code plus config template, setup steps, sample data, tests/evals and known limitations. 'Works on my machine' is not a handoff.

GUIDED LAB - PART I

LAB GOAL

Create a 40-case eval set for one project. Include normal, edge, adversarial and missing-information cases. Track failure category, expected behavior and actual result.

STEP 1 - DEFINE ACCEPTANCE

Write 3-5 observable success conditions before building, including one must-not-happen condition.

STEP 2 - PREPARE TEST DATA

Create normal, edge and malformed examples. Use sandbox/synthetic data.

STEP 3 - BUILD A VERTICAL SLICE

Implement one end-to-end path from trigger/input to validated output. Keep risky side effects mocked or sandboxed first.

STEP 4 - ADD OBSERVABILITY

Record request/workflow ID, timing, decision result, validation result and error category.

STEP 5 - INSPECT INTERMEDIATES

Run manually and inspect each boundary once before adding complexity.

STEP 6 - CAPTURE BASELINE

Record first success rate, latency and estimated cost/task.

STEP 7 - WRITE ACCEPTANCE TESTS

Turn success conditions into repeatable tests/evals, including one case that must escalate.

LAB DELIVERABLE

Commit code/workflow export, sample payloads, architecture image, README and test results.

GUIDED LAB - PART II: HARDENING & DEBUGGING

Test empty input, unexpected type, missing credential, provider timeout, rate limit, low-confidence AI output, invalid structured output, duplicate trigger and downstream API rejection.

DEBUGGING METHOD

1. Reproduce minimally.
2. Locate the failing boundary.
3. Inspect raw input/output safely.
4. Classify: validation, model, network, auth, permission, rate limit, provider, business rule or data.
5. Fix root cause.
6. Add regression test/eval.
7. Document symptom and recovery.

ROOT-CAUSE VS PATCH

Do not add arbitrary delays, broad retries or larger prompts without evidence. Name the root cause and the prevention mechanism.

RECOVERY DESIGN

For each terminal failure choose: retry later, review queue, compensate/rollback, alert operator, or safely discard.

CLIENT-READY FINISH

README sections: Known Limitations and Operations / Recovery.

PORTFOLIO TIP

Include one 'What went wrong' story. A well-explained bug and fix demonstrates seniority better than a perfect demo.

CLIENT SCENARIO

A client says accuracy is "about 90%." Convert that into measurable task-specific criteria and a repeatable acceptance test before production launch.

HOW A SENIOR FREELANCER RESPONDS

Restate the business outcome. Ask about process, volume, systems, sensitive data, permissions, exceptions and cost of mistakes. Propose the smallest phase that proves value. Define AI vs deterministic steps, approvals, acceptance criteria and handoff.

SECURITY / RELIABILITY REVIEW

Check least-privilege credentials, input/output validation, prompt injection exposure, privacy, idempotency, retries, escalation, audit logs and rollback.

SENIOR INTERVIEW PRACTICE

Q1. What is an eval?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q2. How do you test an agent that calls tools?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q3. What is prompt injection?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q4. How do you reduce privacy exposure?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

DISCOVERY QUESTIONS

What happens today? What is volume and error cost? Which APIs are mandatory? Which data is sensitive? Which actions need approval? What metric defines a successful pilot?

PROPOSAL ANGLE

Offer a first milestone that reduces the highest uncertainty: API feasibility, retrieval quality, classification accuracy or workflow reliability.

COMMERCIAL NOTE

Estimate from scope/dependencies/risk. Never promise 100% accuracy or full autonomy when acceptable error handling is undefined.

HOMEWORK & PORTFOLIO OUTPUT

- Build an eval spreadsheet/JSONL dataset.
- Write an AI security threat model.
- Create a pre-production acceptance checklist with measurable thresholds.

PORTFOLIO CHECKLIST

- problem statement
- architecture diagram
- setup instructions
- example input/output
- tests or eval cases
- error handling
- security/permissions note
- known limitations
- demo steps

SUBMISSION RUBRIC

Correctness 30% | Reliability 20% | Security 15% | Tests/evals 15% | Documentation 10% | Architecture reasoning 10%.

CLIENT-FACING SUMMARY

Write five sentences: problem, automated flow, where AI is used, how risk is controlled, and expected measurable result. Reuse this in LinkedIn/portfolio/Upwork material.

SELF-ASSESSMENT

Can you justify the architecture, reproduce a failure, diagnose it, and explain scope/success to a client?

NEXT-DAY BRIDGE

Record one unresolved technical question and one business question.

REFERENCE HABIT

Check current official docs for SDK syntax, model/tool availability, limits and platform behavior before production.

ROLE TARGET

Production AI Engineer

WHY THIS DAY MATTERS

Clients rarely need a model in isolation; they need a dependable business system. Today you will connect the technical topic to architecture, failure handling and a portfolio-ready deliverable.

LEARNING OBJECTIVES

- Deploy AI applications with reproducible configuration.
- Monitor latency, cost, failures and quality signals.
- Design caching, queues and concurrency controls.
- Create maintainable client handoff and operations documentation.

60-MINUTE DELIVERY PLAN

0-5 min: goals/job relevance; 5-15: core concepts; 15-25: architecture; 25-35: implementation; 35-50: guided lab; 50-56: debugging/security; 56-60: interview check + assignment.

END-OF-DAY EVIDENCE

Leave with implementation notes, a tested artifact, interview answers and a client-facing explanation of tradeoffs.

INSTRUCTOR PROMPTS

- Give one business example where this skill creates measurable value.
- Which part should remain deterministic code?
- What data or permissions make the design risky?
- How would you evaluate the result before release?

PRE-LECTURE PREP

Open the project repository, prepare sandbox credentials/test data, review the previous day and write one question the lab should answer.

5-MINUTE CHECK

Explain today's concept without buzzwords, sketch the architecture from memory, and name the most expensive likely failure.

CORE CONCEPTS - PART I

1. Environment configuration. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
2. Docker basics. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
3. Queues. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
4. Async jobs. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
5. Caching. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
6. Concurrency. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
7. Timeouts. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.

TEACHING EMPHASIS

Do not memorize vocabulary. Translate each concept into a design choice and a measurable outcome.

CONCEPT DRILL

Complete: 'I would use this when ____; avoid it when ____; test it by ____.'

COMMON FAILURE PATTERN

Junior implementations often optimize the model call while ignoring validation, permissions, retries, data quality and human exception handling. Those surrounding controls usually determine production success.

SENIOR TAKEAWAY

The more probabilistic the component, the more explicit its measurement and fallback strategy must be.

CORE CONCEPTS - PART II

8. Rate limits. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
9. Usage tracking. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
10. Tracing. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
11. Logs/metrics. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
12. Alerting. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
13. SLOs. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
14. Cost budgets. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

SENIOR-LEVEL CONNECTION

Every probabilistic decision needs validation/evaluation; every side effect needs authorization; every integration needs a failure strategy; every quality claim needs evidence.

TRADEOFF EXERCISE

Compare a fast/cheap implementation with a high-reliability implementation. Discuss cost, latency, complexity, accuracy and failure impact.

CLIENT TRANSLATION

Sell the changed business process, not the technical term: what manual step changes, what evidence is used, what becomes faster, and what a human still approves.

REVIEW QUESTION

If this component were wrong for one week, how would the team notice? If the answer is 'they would not', add observability or an eval checkpoint.

ARCHITECTURE WALKTHROUGH

Reference flow:

Client -> API gateway -> App service -> Queue/workers -> LLM/agent services -> DB/vector store -> Observability -> Secrets manager

1. Client: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
2. API gateway: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
3. App service: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
4. Queue/workers: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
5. LLM/agent services: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
6. DB/vector store: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
7. Observability: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
8. Secrets manager: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.

DESIGN REVIEW QUESTIONS

- Where is untrusted input introduced?
- Where are secrets or sensitive data present?
- Which step is probabilistic?
- Which step changes a system of record?
- What is the human exception path?
- What proves success?

FAILURE-MODE WALKTHROUGH

Test mentally: model timeout, stale data, 429, expired credential, duplicate webhook, unauthorized request. Add a graceful path for relevant failures.

BOUNDARY RULE

Never pass probabilistic output directly into a high-impact side effect. Insert schema validation, business rules, authorization and, where needed, human approval.

DELIVERABLE

Create a technical diagram and a simpler client diagram.

IMPLEMENTATION WALKTHROUGH

Operational metrics:

request_count, success_rate, p50/p95_latency, model_tokens, cost_per_task, tool_error_rate, retry_rate, human_escalation_rate, eval_score, queue_depth

HOW TO READ THIS EXAMPLE

1. Identify component contracts. 2. Separate configuration/secrets. 3. Validate input. 4. Validate model/tool output. 5. Add structured errors and correlation IDs. 6. Prefer explicit schemas over parsing prose.

PRODUCTION EXTENSIONS

Add timeouts, transient retries, rate-limit handling, test fixtures, logging, environment config and a small eval dataset.

TEST PLAN

- happy path
- malformed/missing fields
- timeout / rate-limit
- invalid structured output
- duplicate request
- unauthorized action
- downstream rejection

CODE-REVIEW QUESTIONS

Where can exceptions escape? Which values are trusted? Which calls create side effects? Are secrets loaded safely? Can one request be traced end-to-end? Can prompts/business rules change independently?

OUTSOURCING STANDARD

Deliver code plus config template, setup steps, sample data, tests/evals and known limitations. 'Works on my machine' is not a handoff.

GUIDED LAB - PART I

LAB GOAL

Containerize or document deployment for one project. Add structured logs, request IDs, token/cost tracking, timeout handling and a simple health endpoint.

STEP 1 - DEFINE ACCEPTANCE

Write 3-5 observable success conditions before building, including one must-not-happen condition.

STEP 2 - PREPARE TEST DATA

Create normal, edge and malformed examples. Use sandbox/synthetic data.

STEP 3 - BUILD A VERTICAL SLICE

Implement one end-to-end path from trigger/input to validated output. Keep risky side effects mocked or sandboxed first.

STEP 4 - ADD OBSERVABILITY

Record request/workflow ID, timing, decision result, validation result and error category.

STEP 5 - INSPECT INTERMEDIATES

Run manually and inspect each boundary once before adding complexity.

STEP 6 - CAPTURE BASELINE

Record first success rate, latency and estimated cost/task.

STEP 7 - WRITE ACCEPTANCE TESTS

Turn success conditions into repeatable tests/evals, including one case that must escalate.

LAB DELIVERABLE

Commit code/workflow export, sample payloads, architecture image, README and test results.

GUIDED LAB - PART II: HARDENING & DEBUGGING

Test empty input, unexpected type, missing credential, provider timeout, rate limit, low-confidence AI output, invalid structured output, duplicate trigger and downstream API rejection.

DEBUGGING METHOD

1. Reproduce minimally.
2. Locate the failing boundary.
3. Inspect raw input/output safely.
4. Classify: validation, model, network, auth, permission, rate limit, provider, business rule or data.
5. Fix root cause.
6. Add regression test/eval.
7. Document symptom and recovery.

ROOT-CAUSE VS PATCH

Do not add arbitrary delays, broad retries or larger prompts without evidence. Name the root cause and the prevention mechanism.

RECOVERY DESIGN

For each terminal failure choose: retry later, review queue, compensate/rollback, alert operator, or safely discard.

CLIENT-READY FINISH

README sections: Known Limitations and Operations / Recovery.

PORTFOLIO TIP

Include one 'What went wrong' story. A well-explained bug and fix demonstrates seniority better than a perfect demo.

CLIENT SCENARIO

A prototype works on a laptop but fails under real traffic. Explain the production gaps: secrets, concurrency, queues, timeouts, provider limits, retries, logging and rollback.

HOW A SENIOR FREELANCER RESPONDS

Restate the business outcome. Ask about process, volume, systems, sensitive data, permissions, exceptions and cost of mistakes. Propose the smallest phase that proves value. Define AI vs deterministic steps, approvals, acceptance criteria and handoff.

SECURITY / RELIABILITY REVIEW

Check least-privilege credentials, input/output validation, prompt injection exposure, privacy, idempotency, retries, escalation, audit logs and rollback.

SENIOR INTERVIEW PRACTICE

Q1. What should you monitor in an AI service?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q2. How do you control cost?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q3. When should work be queued asynchronously?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q4. What belongs in a production handoff?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

DISCOVERY QUESTIONS

What happens today? What is volume and error cost? Which APIs are mandatory? Which data is sensitive? Which actions need approval? What metric defines a successful pilot?

PROPOSAL ANGLE

Offer a first milestone that reduces the highest uncertainty: API feasibility, retrieval quality, classification accuracy or workflow reliability.

COMMERCIAL NOTE

Estimate from scope/dependencies/risk. Never promise 100% accuracy or full autonomy when acceptable error handling is undefined.

HOMEWORK & PORTFOLIO OUTPUT

- Create a production architecture diagram.
- Define three SLOs.
- Write a runbook for provider outage, high error rate and unexpected cost spike.

PORTFOLIO CHECKLIST

- problem statement
- architecture diagram
- setup instructions
- example input/output
- tests or eval cases
- error handling
- security/permissions note
- known limitations
- demo steps

SUBMISSION RUBRIC

Correctness 30% | Reliability 20% | Security 15% | Tests/evals 15% | Documentation 10% | Architecture reasoning 10%.

CLIENT-FACING SUMMARY

Write five sentences: problem, automated flow, where AI is used, how risk is controlled, and expected measurable result. Reuse this in LinkedIn/portfolio/Upwork material.

SELF-ASSESSMENT

Can you justify the architecture, reproduce a failure, diagnose it, and explain scope/success to a client?

NEXT-DAY BRIDGE

Record one unresolved technical question and one business question.

REFERENCE HABIT

Check current official docs for SDK syntax, model/tool availability, limits and platform behavior before production.

ROLE TARGET

AI Automation Consultant / Solutions Architect

WHY THIS DAY MATTERS

Clients rarely need a model in isolation; they need a dependable business system. Today you will connect the technical topic to architecture, failure handling and a portfolio-ready deliverable.

LEARNING OBJECTIVES

- Run a structured discovery call.
- Turn vague requirements into measurable scope.
- Estimate work using risks and dependencies.
- Create architecture and acceptance criteria before implementation.

60-MINUTE DELIVERY PLAN

0-5 min: goals/job relevance; 5-15: core concepts; 15-25: architecture; 25-35: implementation; 35-50: guided lab; 50-56: debugging/security; 56-60: interview check + assignment.

END-OF-DAY EVIDENCE

Leave with implementation notes, a tested artifact, interview answers and a client-facing explanation of tradeoffs.

INSTRUCTOR PROMPTS

- Give one business example where this skill creates measurable value.
- Which part should remain deterministic code?
- What data or permissions make the design risky?
- How would you evaluate the result before release?

PRE-LECTURE PREP

Open the project repository, prepare sandbox credentials/test data, review the previous day and write one question the lab should answer.

5-MINUTE CHECK

Explain today's concept without buzzwords, sketch the architecture from memory, and name the most expensive likely failure.

CORE CONCEPTS - PART I

1. Discovery questions. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
2. Current-state process. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
3. Pain/volume/value. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
4. Data sources. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
5. Systems and APIs. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
6. Security constraints. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.

TEACHING EMPHASIS

Do not memorize vocabulary. Translate each concept into a design choice and a measurable outcome.

CONCEPT DRILL

Complete: 'I would use this when ____; avoid it when ____; test it by ____.'

COMMON FAILURE PATTERN

Junior implementations often optimize the model call while ignoring validation, permissions, retries, data quality and human exception handling. Those surrounding controls usually determine production success.

SENIOR TAKEAWAY

The more probabilistic the component, the more explicit its measurement and fallback strategy must be.

CORE CONCEPTS - PART II

7. Acceptance criteria. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

8. Milestones. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

9. Risks. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

10. Assumptions. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

11. Change control. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

SENIOR-LEVEL CONNECTION

Every probabilistic decision needs validation/evaluation; every side effect needs authorization; every integration needs a failure strategy; every quality claim needs evidence.

TRADEOFF EXERCISE

Compare a fast/cheap implementation with a high-reliability implementation. Discuss cost, latency, complexity, accuracy and failure impact.

CLIENT TRANSLATION

Sell the changed business process, not the technical term: what manual step changes, what evidence is used, what becomes faster, and what a human still approves.

REVIEW QUESTION

If this component were wrong for one week, how would the team notice? If the answer is 'they would not', add observability or an eval checkpoint.

ARCHITECTURE WALKTHROUGH

Reference flow:

Discovery -> Requirements -> Process map -> Architecture options -> Prototype -> Acceptance tests -> Production plan -> Handoff

1. Discovery: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
2. Requirements: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
3. Process map: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
4. Architecture options: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
5. Prototype: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
6. Acceptance tests: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
7. Production plan: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
8. Handoff: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.

DESIGN REVIEW QUESTIONS

- Where is untrusted input introduced?
- Where are secrets or sensitive data present?
- Which step is probabilistic?
- Which step changes a system of record?
- What is the human exception path?
- What proves success?

FAILURE-MODE WALKTHROUGH

Test mentally: model timeout, stale data, 429, expired credential, duplicate webhook, unauthorized request. Add a graceful path for relevant failures.

BOUNDARY RULE

Never pass probabilistic output directly into a high-impact side effect. Insert schema validation, business rules, authorization and, where needed, human approval.

DELIVERABLE

Create a technical diagram and a simpler client diagram.

IMPLEMENTATION WALKTHROUGH

Scope statement formula:

For [users], automate [process] from [trigger] to [outcome] using [systems], with [human approvals], measured by [KPIs].

Exclude [out-of-scope items].

HOW TO READ THIS EXAMPLE

1. Identify component contracts. 2. Separate configuration/secrets. 3. Validate input. 4. Validate model/tool output. 5. Add structured errors and correlation IDs. 6. Prefer explicit schemas over parsing prose.

PRODUCTION EXTENSIONS

Add timeouts, transient retries, rate-limit handling, test fixtures, logging, environment config and a small eval dataset.

TEST PLAN

- happy path
- malformed/missing fields
- timeout / rate-limit
- invalid structured output
- duplicate request
- unauthorized action
- downstream rejection

CODE-REVIEW QUESTIONS

Where can exceptions escape? Which values are trusted? Which calls create side effects? Are secrets loaded safely? Can one request be traced end-to-end? Can prompts/business rules change independently?

OUTSOURCING STANDARD

Deliver code plus config template, setup steps, sample data, tests/evals and known limitations. 'Works on my machine' is not a handoff.

GUIDED LAB - PART I

LAB GOAL

Take a vague brief: "Build an AI employee for my business." Produce discovery questions, assumptions, a phased architecture, acceptance criteria and a fixed-scope Phase 1.

STEP 1 - DEFINE ACCEPTANCE

Write 3-5 observable success conditions before building, including one must-not-happen condition.

STEP 2 - PREPARE TEST DATA

Create normal, edge and malformed examples. Use sandbox/synthetic data.

STEP 3 - BUILD A VERTICAL SLICE

Implement one end-to-end path from trigger/input to validated output. Keep risky side effects mocked or sandboxed first.

STEP 4 - ADD OBSERVABILITY

Record request/workflow ID, timing, decision result, validation result and error category.

STEP 5 - INSPECT INTERMEDIATES

Run manually and inspect each boundary once before adding complexity.

STEP 6 - CAPTURE BASELINE

Record first success rate, latency and estimated cost/task.

STEP 7 - WRITE ACCEPTANCE TESTS

Turn success conditions into repeatable tests/evals, including one case that must escalate.

LAB DELIVERABLE

Commit code/workflow export, sample payloads, architecture image, README and test results.

GUIDED LAB - PART II: HARDENING & DEBUGGING

Test empty input, unexpected type, missing credential, provider timeout, rate limit, low-confidence AI output, invalid structured output, duplicate trigger and downstream API rejection.

DEBUGGING METHOD

1. Reproduce minimally.
2. Locate the failing boundary.
3. Inspect raw input/output safely.
4. Classify: validation, model, network, auth, permission, rate limit, provider, business rule or data.
5. Fix root cause.
6. Add regression test/eval.
7. Document symptom and recovery.

ROOT-CAUSE VS PATCH

Do not add arbitrary delays, broad retries or larger prompts without evidence. Name the root cause and the prevention mechanism.

RECOVERY DESIGN

For each terminal failure choose: retry later, review queue, compensate/rollback, alert operator, or safely discard.

CLIENT-READY FINISH

README sections: Known Limitations and Operations / Recovery.

PORTFOLIO TIP

Include one 'What went wrong' story. A well-explained bug and fix demonstrates seniority better than a perfect demo.

CLIENT SCENARIO

The client wants Gmail, CRM, Slack, documents, forecasting and autonomous follow-up in one milestone. Break the work into value-tested phases and explain dependency risk.

HOW A SENIOR FREELANCER RESPONDS

Restate the business outcome. Ask about process, volume, systems, sensitive data, permissions, exceptions and cost of mistakes. Propose the smallest phase that proves value. Define AI vs deterministic steps, approvals, acceptance criteria and handoff.

SECURITY / RELIABILITY REVIEW

Check least-privilege credentials, input/output validation, prompt injection exposure, privacy, idempotency, retries, escalation, audit logs and rollback.

SENIOR INTERVIEW PRACTICE

Q1. What questions do you ask before estimating?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q2. How do you handle unknown API limitations?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q3. What makes acceptance criteria testable?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q4. How do you protect scope from uncontrolled expansion?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

DISCOVERY QUESTIONS

What happens today? What is volume and error cost? Which APIs are mandatory? Which data is sensitive? Which actions need approval? What metric defines a successful pilot?

PROPOSAL ANGLE

Offer a first milestone that reduces the highest uncertainty: API feasibility, retrieval quality, classification accuracy or workflow reliability.

COMMERCIAL NOTE

Estimate from scope/dependencies/risk. Never promise 100% accuracy or full autonomy when acceptable error handling is undefined.

HOMEWORK & PORTFOLIO OUTPUT

- Create a reusable discovery questionnaire.
- Write a sample statement of work.
- Estimate one capstone project in milestones with assumptions and risks.

PORTFOLIO CHECKLIST

- problem statement
- architecture diagram
- setup instructions
- example input/output
- tests or eval cases
- error handling
- security/permissions note
- known limitations
- demo steps

SUBMISSION RUBRIC

Correctness 30% | Reliability 20% | Security 15% | Tests/evals 15% | Documentation 10% | Architecture reasoning 10%.

CLIENT-FACING SUMMARY

Write five sentences: problem, automated flow, where AI is used, how risk is controlled, and expected measurable result. Reuse this in LinkedIn/portfolio/Upwork material.

SELF-ASSESSMENT

Can you justify the architecture, reproduce a failure, diagnose it, and explain scope/success to a client?

NEXT-DAY BRIDGE

Record one unresolved technical question and one business question.

REFERENCE HABIT

Check current official docs for SDK syntax, model/tool availability, limits and platform behavior before production.

ROLE TARGET

Freelance AI Automation Specialist

WHY THIS DAY MATTERS

Clients rarely need a model in isolation; they need a dependable business system. Today you will connect the technical topic to architecture, failure handling and a portfolio-ready deliverable.

LEARNING OBJECTIVES

- Position services around business outcomes and technical credibility.
- Write high-signal proposals tied to a client brief.
- Present portfolio projects as case studies.
- Prepare for technical and client-facing interviews.

60-MINUTE DELIVERY PLAN

0-5 min: goals/job relevance; 5-15: core concepts; 15-25: architecture; 25-35: implementation; 35-50: guided lab; 50-56: debugging/security; 56-60: interview check + assignment.

END-OF-DAY EVIDENCE

Leave with implementation notes, a tested artifact, interview answers and a client-facing explanation of tradeoffs.

INSTRUCTOR PROMPTS

- Give one business example where this skill creates measurable value.
- Which part should remain deterministic code?
- What data or permissions make the design risky?
- How would you evaluate the result before release?

PRE-LECTURE PREP

Open the project repository, prepare sandbox credentials/test data, review the previous day and write one question the lab should answer.

5-MINUTE CHECK

Explain today's concept without buzzwords, sketch the architecture from memory, and name the most expensive likely failure.

CORE CONCEPTS - PART I

1. Niche positioning. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
2. Outcome-based profile. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
3. Portfolio evidence. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
4. Proposal structure. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
5. Risk-reversal. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.

TEACHING EMPHASIS

Do not memorize vocabulary. Translate each concept into a design choice and a measurable outcome.

CONCEPT DRILL

Complete: 'I would use this when ____; avoid it when ____; test it by ____.'

COMMON FAILURE PATTERN

Junior implementations often optimize the model call while ignoring validation, permissions, retries, data quality and human exception handling. Those surrounding controls usually determine production success.

SENIOR TAKEAWAY

The more probabilistic the component, the more explicit its measurement and fallback strategy must be.

CORE CONCEPTS - PART II

6. Discovery call. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
7. Technical screening. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
8. Architecture explanation. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
9. Pricing models. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
10. Maintenance retainers. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

SENIOR-LEVEL CONNECTION

Every probabilistic decision needs validation/evaluation; every side effect needs authorization; every integration needs a failure strategy; every quality claim needs evidence.

TRADEOFF EXERCISE

Compare a fast/cheap implementation with a high-reliability implementation. Discuss cost, latency, complexity, accuracy and failure impact.

CLIENT TRANSLATION

Sell the changed business process, not the technical term: what manual step changes, what evidence is used, what becomes faster, and what a human still approves.

REVIEW QUESTION

If this component were wrong for one week, how would the team notice? If the answer is 'they would not', add observability or an eval checkpoint.

ARCHITECTURE WALKTHROUGH

Reference flow:

Job brief -> Requirement extraction -> Relevant proof -> Mini solution outline -> Questions/risks -> Proposal -> Discovery -> Paid milestone

1. Job brief: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
2. Requirement extraction: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
3. Relevant proof: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
4. Mini solution outline: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
5. Questions/risks: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
6. Proposal: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
7. Discovery: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
8. Paid milestone: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.

DESIGN REVIEW QUESTIONS

- Where is untrusted input introduced?
- Where are secrets or sensitive data present?
- Which step is probabilistic?
- Which step changes a system of record?
- What is the human exception path?
- What proves success?

FAILURE-MODE WALKTHROUGH

Test mentally: model timeout, stale data, 429, expired credential, duplicate webhook, unauthorized request. Add a graceful path for relevant failures.

BOUNDARY RULE

Never pass probabilistic output directly into a high-impact side effect. Insert schema validation, business rules, authorization and, where needed, human approval.

DELIVERABLE

Create a technical diagram and a simpler client diagram.

IMPLEMENTATION WALKTHROUGH

Proposal skeleton:

1. Restate the business outcome.
2. Identify the likely architecture.
3. Mention one relevant proof/project.
4. Flag one real risk or dependency.
5. Propose a small first milestone.
6. Ask 2-3 sharp discovery questions.

HOW TO READ THIS EXAMPLE

1. Identify component contracts. 2. Separate configuration/secrets. 3. Validate input. 4. Validate model/tool output. 5. Add structured errors and correlation IDs. 6. Prefer explicit schemas over parsing prose.

PRODUCTION EXTENSIONS

Add timeouts, transient retries, rate-limit handling, test fixtures, logging, environment config and a small eval dataset.

TEST PLAN

- happy path
- malformed/missing fields
- timeout / rate-limit
- invalid structured output
- duplicate request
- unauthorized action
- downstream rejection

CODE-REVIEW QUESTIONS

Where can exceptions escape? Which values are trusted? Which calls create side effects? Are secrets loaded safely? Can one request be traced end-to-end? Can prompts/business rules change independently?

OUTSOURCING STANDARD

Deliver code plus config template, setup steps, sample data, tests/evals and known limitations. 'Works on my machine' is not a handoff.

GUIDED LAB - PART I

LAB GOAL

Write three proposal variants for: RAG knowledge assistant, n8n lead automation, and AI agent with API tools. Each must be specific, short and technically credible.

STEP 1 - DEFINE ACCEPTANCE

Write 3-5 observable success conditions before building, including one must-not-happen condition.

STEP 2 - PREPARE TEST DATA

Create normal, edge and malformed examples. Use sandbox/synthetic data.

STEP 3 - BUILD A VERTICAL SLICE

Implement one end-to-end path from trigger/input to validated output. Keep risky side effects mocked or sandboxed first.

STEP 4 - ADD OBSERVABILITY

Record request/workflow ID, timing, decision result, validation result and error category.

STEP 5 - INSPECT INTERMEDIATES

Run manually and inspect each boundary once before adding complexity.

STEP 6 - CAPTURE BASELINE

Record first success rate, latency and estimated cost/task.

STEP 7 - WRITE ACCEPTANCE TESTS

Turn success conditions into repeatable tests/evals, including one case that must escalate.

LAB DELIVERABLE

Commit code/workflow export, sample payloads, architecture image, README and test results.

GUIDED LAB - PART II: HARDENING & DEBUGGING

Test empty input, unexpected type, missing credential, provider timeout, rate limit, low-confidence AI output, invalid structured output, duplicate trigger and downstream API rejection.

DEBUGGING METHOD

1. Reproduce minimally.
2. Locate the failing boundary.
3. Inspect raw input/output safely.
4. Classify: validation, model, network, auth, permission, rate limit, provider, business rule or data.
5. Fix root cause.
6. Add regression test/eval.
7. Document symptom and recovery.

ROOT-CAUSE VS PATCH

Do not add arbitrary delays, broad retries or larger prompts without evidence. Name the root cause and the prevention mechanism.

RECOVERY DESIGN

For each terminal failure choose: retry later, review queue, compensate/rollback, alert operator, or safely discard.

CLIENT-READY FINISH

README sections: Known Limitations and Operations / Recovery.

PORTFOLIO TIP

Include one 'What went wrong' story. A well-explained bug and fix demonstrates seniority better than a perfect demo.

CLIENT SCENARIO

A job post asks for "ChatGPT + n8n expert." Demonstrate seniority by discussing workflow reliability, permissions, evaluation and handoff—not by listing dozens of tools.

HOW A SENIOR FREELANCER RESPONDS

Restate the business outcome. Ask about process, volume, systems, sensitive data, permissions, exceptions and cost of mistakes. Propose the smallest phase that proves value. Define AI vs deterministic steps, approvals, acceptance criteria and handoff.

SECURITY / RELIABILITY REVIEW

Check least-privilege credentials, input/output validation, prompt injection exposure, privacy, idempotency, retries, escalation, audit logs and rollback.

SENIOR INTERVIEW PRACTICE

Q1. How would you scope an AI automation project in 30 minutes?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q2. What portfolio evidence is strongest?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q3. How do you explain AI limitations to a client?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q4. When do you recommend a retainer?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

DISCOVERY QUESTIONS

What happens today? What is volume and error cost? Which APIs are mandatory? Which data is sensitive? Which actions need approval? What metric defines a successful pilot?

PROPOSAL ANGLE

Offer a first milestone that reduces the highest uncertainty: API feasibility, retrieval quality, classification accuracy or workflow reliability.

COMMERCIAL NOTE

Estimate from scope/dependencies/risk. Never promise 100% accuracy or full autonomy when acceptable error handling is undefined.

HOMEWORK & PORTFOLIO OUTPUT

- Rewrite your LinkedIn headline and About section.
- Create three portfolio case-study pages.
- Prepare answers to 25 technical and consulting questions from this course.

PORTFOLIO CHECKLIST

- problem statement
- architecture diagram
- setup instructions
- example input/output
- tests or eval cases
- error handling
- security/permissions note
- known limitations
- demo steps

SUBMISSION RUBRIC

Correctness 30% | Reliability 20% | Security 15% | Tests/evals 15% | Documentation 10% | Architecture reasoning 10%.

CLIENT-FACING SUMMARY

Write five sentences: problem, automated flow, where AI is used, how risk is controlled, and expected measurable result. Reuse this in LinkedIn/portfolio/Upwork material.

SELF-ASSESSMENT

Can you justify the architecture, reproduce a failure, diagnose it, and explain scope/success to a client?

NEXT-DAY BRIDGE

Record one unresolved technical question and one business question.

REFERENCE HABIT

Check current official docs for SDK syntax, model/tool availability, limits and platform behavior before production.

ROLE TARGET

Full-Stack AI Automation Architect

WHY THIS DAY MATTERS

Clients rarely need a model in isolation; they need a dependable business system. Today you will connect the technical topic to architecture, failure handling and a portfolio-ready deliverable.

LEARNING OBJECTIVES

- Combine API, RAG, workflow automation, tools and evaluation.
- Build an end-to-end system with clear boundaries.
- Produce professional technical documentation.
- Demonstrate a portfolio project that resembles real outsourcing work.

60-MINUTE DELIVERY PLAN

0-5 min: goals/job relevance; 5-15: core concepts; 15-25: architecture; 25-35: implementation; 35-50: guided lab; 50-56: debugging/security; 56-60: interview check + assignment.

END-OF-DAY EVIDENCE

Leave with implementation notes, a tested artifact, interview answers and a client-facing explanation of tradeoffs.

INSTRUCTOR PROMPTS

- Give one business example where this skill creates measurable value.
- Which part should remain deterministic code?
- What data or permissions make the design risky?
- How would you evaluate the result before release?

PRE-LECTURE PREP

Open the project repository, prepare sandbox credentials/test data, review the previous day and write one question the lab should answer.

5-MINUTE CHECK

Explain today's concept without buzzwords, sketch the architecture from memory, and name the most expensive likely failure.

CORE CONCEPTS - PART I

1. Capstone architecture. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
2. Customer support triage. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
3. Knowledge retrieval. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
4. Lead qualification. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
5. CRM update. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.

TEACHING EMPHASIS

Do not memorize vocabulary. Translate each concept into a design choice and a measurable outcome.

CONCEPT DRILL

Complete: 'I would use this when ____; avoid it when ____; test it by ____.'

COMMON FAILURE PATTERN

Junior implementations often optimize the model call while ignoring validation, permissions, retries, data quality and human exception handling. Those surrounding controls usually determine production success.

SENIOR TAKEAWAY

The more probabilistic the component, the more explicit its measurement and fallback strategy must be.

CORE CONCEPTS - PART II

6. Reporting. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
7. Approvals. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
8. Monitoring. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
9. Evaluation. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
10. Deployment. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

SENIOR-LEVEL CONNECTION

Every probabilistic decision needs validation/evaluation; every side effect needs authorization; every integration needs a failure strategy; every quality claim needs evidence.

TRADEOFF EXERCISE

Compare a fast/cheap implementation with a high-reliability implementation. Discuss cost, latency, complexity, accuracy and failure impact.

CLIENT TRANSLATION

Sell the changed business process, not the technical term: what manual step changes, what evidence is used, what becomes faster, and what a human still approves.

REVIEW QUESTION

If this component were wrong for one week, how would the team notice? If the answer is 'they would not', add observability or an eval checkpoint.

ARCHITECTURE WALKTHROUGH

Reference flow:

Email/webhook inputs -> n8n orchestration -> LLM classification -> RAG knowledge base -> CRM/helpdesk tools -> Human approvals -> Database/logs -> Dashboard/reporting

1. Email/webhook inputs: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
2. n8n orchestration: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
3. LLM classification: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
4. RAG knowledge base: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
5. CRM/helpdesk tools: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
6. Human approvals: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
7. Database/logs: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
8. Dashboard/reporting: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.

DESIGN REVIEW QUESTIONS

- Where is untrusted input introduced?
- Where are secrets or sensitive data present?
- Which step is probabilistic?
- Which step changes a system of record?
- What is the human exception path?
- What proves success?

FAILURE-MODE WALKTHROUGH

Test mentally: model timeout, stale data, 429, expired credential, duplicate webhook, unauthorized request. Add a graceful path for relevant failures.

BOUNDARY RULE

Never pass probabilistic output directly into a high-impact side effect. Insert schema validation, business rules, authorization and, where needed, human approval.

DELIVERABLE

Create a technical diagram and a simpler client diagram.

IMPLEMENTATION WALKTHROUGH

Capstone acceptance tests:

- Support requests routed correctly
- Knowledge answers cite source
- Leads create valid CRM records
- No write action without policy/approval
- Failures enter review queue
- Logs include correlation ID
- Eval suite passes agreed thresholds

HOW TO READ THIS EXAMPLE

1. Identify component contracts. 2. Separate configuration/secrets. 3. Validate input. 4. Validate model/tool output. 5. Add structured errors and correlation IDs. 6. Prefer explicit schemas over parsing prose.

PRODUCTION EXTENSIONS

Add timeouts, transient retries, rate-limit handling, test fixtures, logging, environment config and a small eval dataset.

TEST PLAN

- happy path
- malformed/missing fields
- timeout / rate-limit
- invalid structured output
- duplicate request
- unauthorized action
- downstream rejection

CODE-REVIEW QUESTIONS

Where can exceptions escape? Which values are trusted? Which calls create side effects? Are secrets loaded safely? Can one request be traced end-to-end? Can prompts/business rules change independently?

OUTSOURCING STANDARD

Deliver code plus config template, setup steps, sample data, tests/evals and known limitations. 'Works on my machine' is not a handoff.

GUIDED LAB - PART I

LAB GOAL

Build the capstone in vertical slices: intake -> classify -> retrieve -> draft -> approval -> system update -> log. Demonstrate at least one failure path and one human-review path.

STEP 1 - DEFINE ACCEPTANCE

Write 3-5 observable success conditions before building, including one must-not-happen condition.

STEP 2 - PREPARE TEST DATA

Create normal, edge and malformed examples. Use sandbox/synthetic data.

STEP 3 - BUILD A VERTICAL SLICE

Implement one end-to-end path from trigger/input to validated output. Keep risky side effects mocked or sandboxed first.

STEP 4 - ADD OBSERVABILITY

Record request/workflow ID, timing, decision result, validation result and error category.

STEP 5 - INSPECT INTERMEDIATES

Run manually and inspect each boundary once before adding complexity.

STEP 6 - CAPTURE BASELINE

Record first success rate, latency and estimated cost/task.

STEP 7 - WRITE ACCEPTANCE TESTS

Turn success conditions into repeatable tests/evals, including one case that must escalate.

LAB DELIVERABLE

Commit code/workflow export, sample payloads, architecture image, README and test results.

GUIDED LAB - PART II: HARDENING & DEBUGGING

Test empty input, unexpected type, missing credential, provider timeout, rate limit, low-confidence AI output, invalid structured output, duplicate trigger and downstream API rejection.

DEBUGGING METHOD

1. Reproduce minimally.
2. Locate the failing boundary.
3. Inspect raw input/output safely.
4. Classify: validation, model, network, auth, permission, rate limit, provider, business rule or data.
5. Fix root cause.
6. Add regression test/eval.
7. Document symptom and recovery.

ROOT-CAUSE VS PATCH

Do not add arbitrary delays, broad retries or larger prompts without evidence. Name the root cause and the prevention mechanism.

RECOVERY DESIGN

For each terminal failure choose: retry later, review queue, compensate/rollback, alert operator, or safely discard.

CLIENT-READY FINISH

README sections: Known Limitations and Operations / Recovery.

PORTFOLIO TIP

Include one 'What went wrong' story. A well-explained bug and fix demonstrates seniority better than a perfect demo.

CLIENT SCENARIO

Present the capstone as if the evaluator is a paying client. Lead with the manual process, measurable outcome, architecture, demo, controls, limitations and next phase.

HOW A SENIOR FREELANCER RESPONDS

Restate the business outcome. Ask about process, volume, systems, sensitive data, permissions, exceptions and cost of mistakes. Propose the smallest phase that proves value. Define AI vs deterministic steps, approvals, acceptance criteria and handoff.

SECURITY / RELIABILITY REVIEW

Check least-privilege credentials, input/output validation, prompt injection exposure, privacy, idempotency, retries, escalation, audit logs and rollback.

SENIOR INTERVIEW PRACTICE

Q1. Why did you choose workflow vs agent for each step?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q2. Where are the security boundaries?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q3. How do you measure quality?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q4. How would you scale this to multiple departments?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

DISCOVERY QUESTIONS

What happens today? What is volume and error cost? Which APIs are mandatory? Which data is sensitive? Which actions need approval? What metric defines a successful pilot?

PROPOSAL ANGLE

Offer a first milestone that reduces the highest uncertainty: API feasibility, retrieval quality, classification accuracy or workflow reliability.

COMMERCIAL NOTE

Estimate from scope/dependencies/risk. Never promise 100% accuracy or full autonomy when acceptable error handling is undefined.

HOMEWORK & PORTFOLIO OUTPUT

- Complete README, architecture diagram, setup, demo script, eval report, threat model and handoff notes.
- Record a concise demo video or prepare a live demo checklist.
- Create a one-page case study with before/after workflow and measurable KPIs.

PORTFOLIO CHECKLIST

- problem statement
- architecture diagram
- setup instructions
- example input/output
- tests or eval cases
- error handling
- security/permissions note
- known limitations
- demo steps

SUBMISSION RUBRIC

Correctness 30% | Reliability 20% | Security 15% | Tests/evals 15% | Documentation 10% | Architecture reasoning 10%.

CLIENT-FACING SUMMARY

Write five sentences: problem, automated flow, where AI is used, how risk is controlled, and expected measurable result. Reuse this in LinkedIn/portfolio/Upwork material.

SELF-ASSESSMENT

Can you justify the architecture, reproduce a failure, diagnose it, and explain scope/success to a client?

NEXT-DAY BRIDGE

Record one unresolved technical question and one business question.

REFERENCE HABIT

Check current official docs for SDK syntax, model/tool availability, limits and platform behavior before production.

ROLE TARGET

Senior AI Automation Consultant

WHY THIS DAY MATTERS

Clients rarely need a model in isolation; they need a dependable business system. Today you will connect the technical topic to architecture, failure handling and a portfolio-ready deliverable.

LEARNING OBJECTIVES

- Deliver projects so clients can operate them after handoff.
- Create maintenance, monitoring and change-management plans.
- Package reusable assets and lessons learned.
- Plan the next 90 days toward senior-level outsourcing capability.

60-MINUTE DELIVERY PLAN

0-5 min: goals/job relevance; 5-15: core concepts; 15-25: architecture; 25-35: implementation; 35-50: guided lab; 50-56: debugging/security; 56-60: interview check + assignment.

END-OF-DAY EVIDENCE

Leave with implementation notes, a tested artifact, interview answers and a client-facing explanation of tradeoffs.

INSTRUCTOR PROMPTS

- Give one business example where this skill creates measurable value.
- Which part should remain deterministic code?
- What data or permissions make the design risky?
- How would you evaluate the result before release?

PRE-LECTURE PREP

Open the project repository, prepare sandbox credentials/test data, review the previous day and write one question the lab should answer.

5-MINUTE CHECK

Explain today's concept without buzzwords, sketch the architecture from memory, and name the most expensive likely failure.

CORE CONCEPTS - PART I

1. Client handoff. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
2. Runbooks. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
3. Admin documentation. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
4. Credential ownership. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
5. Monitoring. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.
6. Change requests. Define it in plain language. Identify input, output, assumptions, a failure mode, and the signal that would reveal failure. Then connect the concept to one client use case and one case where it is unnecessary.

TEACHING EMPHASIS

Do not memorize vocabulary. Translate each concept into a design choice and a measurable outcome.

CONCEPT DRILL

Complete: 'I would use this when ____; avoid it when ____; test it by ____.'

COMMON FAILURE PATTERN

Junior implementations often optimize the model call while ignoring validation, permissions, retries, data quality and human exception handling. Those surrounding controls usually determine production success.

SENIOR TAKEAWAY

The more probabilistic the component, the more explicit its measurement and fallback strategy must be.

CORE CONCEPTS - PART II

7. Model/provider changes. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
8. Regression testing. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
9. Maintenance retainers. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
10. Portfolio iteration. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.
11. Specialization. Treat this as an engineering control, not a buzzword. State the operational cost, effect on user experience, interaction with security/data quality, and what happens if the component is unavailable or wrong.

SENIOR-LEVEL CONNECTION

Every probabilistic decision needs validation/evaluation; every side effect needs authorization; every integration needs a failure strategy; every quality claim needs evidence.

TRADEOFF EXERCISE

Compare a fast/cheap implementation with a high-reliability implementation. Discuss cost, latency, complexity, accuracy and failure impact.

CLIENT TRANSLATION

Sell the changed business process, not the technical term: what manual step changes, what evidence is used, what becomes faster, and what a human still approves.

REVIEW QUESTION

If this component were wrong for one week, how would the team notice? If the answer is 'they would not', add observability or an eval checkpoint.

ARCHITECTURE WALKTHROUGH

Reference flow:

Build -> Acceptance -> Production release -> Training -> Handoff -> Monitoring -> Maintenance -> Quarterly optimization

1. Build: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
2. Acceptance: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
3. Production release: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
4. Training: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
5. Handoff: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
6. Monitoring: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
7. Maintenance: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.
8. Quarterly optimization: define one responsibility. Specify inputs/outputs, required credentials, state, telemetry and failure behavior. If it can trigger a side effect, add validation and rollback/compensation.

DESIGN REVIEW QUESTIONS

- Where is untrusted input introduced?
- Where are secrets or sensitive data present?
- Which step is probabilistic?
- Which step changes a system of record?
- What is the human exception path?
- What proves success?

FAILURE-MODE WALKTHROUGH

Test mentally: model timeout, stale data, 429, expired credential, duplicate webhook, unauthorized request. Add a graceful path for relevant failures.

BOUNDARY RULE

Never pass probabilistic output directly into a high-impact side effect. Insert schema validation, business rules, authorization and, where needed, human approval.

DELIVERABLE

Create a technical diagram and a simpler client diagram.

IMPLEMENTATION WALKTHROUGH

Handoff package:

README + architecture + workflow exports + env template + credential ownership map + deployment steps + test/eval suite + runbooks + rollback plan + known limitations + maintenance schedule

HOW TO READ THIS EXAMPLE

1. Identify component contracts. 2. Separate configuration/secrets. 3. Validate input. 4. Validate model/tool output. 5. Add structured errors and correlation IDs. 6. Prefer explicit schemas over parsing prose.

PRODUCTION EXTENSIONS

Add timeouts, transient retries, rate-limit handling, test fixtures, logging, environment config and a small eval dataset.

TEST PLAN

- happy path
- malformed/missing fields
- timeout / rate-limit
- invalid structured output
- duplicate request
- unauthorized action
- downstream rejection

CODE-REVIEW QUESTIONS

Where can exceptions escape? Which values are trusted? Which calls create side effects? Are secrets loaded safely? Can one request be traced end-to-end? Can prompts/business rules change independently?

OUTSOURCING STANDARD

Deliver code plus config template, setup steps, sample data, tests/evals and known limitations. 'Works on my machine' is not a handoff.

GUIDED LAB - PART I

LAB GOAL

Perform a mock client handoff for the capstone. Another person should be able to deploy, test, troubleshoot and change one prompt without your verbal help.

STEP 1 - DEFINE ACCEPTANCE

Write 3-5 observable success conditions before building, including one must-not-happen condition.

STEP 2 - PREPARE TEST DATA

Create normal, edge and malformed examples. Use sandbox/synthetic data.

STEP 3 - BUILD A VERTICAL SLICE

Implement one end-to-end path from trigger/input to validated output. Keep risky side effects mocked or sandboxed first.

STEP 4 - ADD OBSERVABILITY

Record request/workflow ID, timing, decision result, validation result and error category.

STEP 5 - INSPECT INTERMEDIATES

Run manually and inspect each boundary once before adding complexity.

STEP 6 - CAPTURE BASELINE

Record first success rate, latency and estimated cost/task.

STEP 7 - WRITE ACCEPTANCE TESTS

Turn success conditions into repeatable tests/evals, including one case that must escalate.

LAB DELIVERABLE

Commit code/workflow export, sample payloads, architecture image, README and test results.

GUIDED LAB - PART II: HARDENING & DEBUGGING

Test empty input, unexpected type, missing credential, provider timeout, rate limit, low-confidence AI output, invalid structured output, duplicate trigger and downstream API rejection.

DEBUGGING METHOD

1. Reproduce minimally.
2. Locate the failing boundary.
3. Inspect raw input/output safely.
4. Classify: validation, model, network, auth, permission, rate limit, provider, business rule or data.
5. Fix root cause.
6. Add regression test/eval.
7. Document symptom and recovery.

ROOT-CAUSE VS PATCH

Do not add arbitrary delays, broad retries or larger prompts without evidence. Name the root cause and the prevention mechanism.

RECOVERY DESIGN

For each terminal failure choose: retry later, review queue, compensate/rollback, alert operator, or safely discard.

CLIENT-READY FINISH

README sections: Known Limitations and Operations / Recovery.

PORTFOLIO TIP

Include one 'What went wrong' story. A well-explained bug and fix demonstrates seniority better than a perfect demo.

CLIENT SCENARIO

A client approves the project and asks, "What happens when OpenAI/n8n/API behavior changes?" Propose monitoring, regression tests, versioning and a maintenance agreement.

HOW A SENIOR FREELANCER RESPONDS

Restate the business outcome. Ask about process, volume, systems, sensitive data, permissions, exceptions and cost of mistakes. Propose the smallest phase that proves value. Define AI vs deterministic steps, approvals, acceptance criteria and handoff.

SECURITY / RELIABILITY REVIEW

Check least-privilege credentials, input/output validation, prompt injection exposure, privacy, idempotency, retries, escalation, audit logs and rollback.

SENIOR INTERVIEW PRACTICE

Q1. What makes a handoff professional?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q2. How do you manage AI system changes over time?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q3. What should be under version control?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

Q4. How do you continue growing from mid-level to senior?

Answer framework: define -> architecture implication -> failure mode -> production control/metric.

DISCOVERY QUESTIONS

What happens today? What is volume and error cost? Which APIs are mandatory? Which data is sensitive? Which actions need approval? What metric defines a successful pilot?

PROPOSAL ANGLE

Offer a first milestone that reduces the highest uncertainty: API feasibility, retrieval quality, classification accuracy or workflow reliability.

COMMERCIAL NOTE

Estimate from scope/dependencies/risk. Never promise 100% accuracy or full autonomy when acceptable error handling is undefined.

HOMEWORK & PORTFOLIO OUTPUT

- Publish/refine your portfolio.
- Create a 90-day specialization plan.
- Apply to or analyze relevant AI automation contracts weekly and track gaps between requirements and your current skills.

PORTFOLIO CHECKLIST

- problem statement
- architecture diagram
- setup instructions
- example input/output
- tests or eval cases
- error handling
- security/permissions note
- known limitations
- demo steps

SUBMISSION RUBRIC

Correctness 30% | Reliability 20% | Security 15% | Tests/evals 15% | Documentation 10% | Architecture reasoning 10%.

CLIENT-FACING SUMMARY

Write five sentences: problem, automated flow, where AI is used, how risk is controlled, and expected measurable result. Reuse this in LinkedIn/portfolio/Upwork material.

SELF-ASSESSMENT

Can you justify the architecture, reproduce a failure, diagnose it, and explain scope/success to a client?

NEXT-DAY BRIDGE

Record one unresolved technical question and one business question.

REFERENCE HABIT

Check current official docs for SDK syntax, model/tool availability, limits and platform behavior before production.

A production prompt is an interface contract. Use a stable structure: role/context -> task -> constraints -> evidence -> output schema -> uncertainty behavior -> examples.

Useful patterns:

- Extract: identify fields from source text; use null for missing evidence.
- Classify: provide category definitions and boundary examples.
- Transform: preserve facts while changing format/tone.
- Critique: compare output against a rubric; do not silently rewrite until asked.
- Plan: decompose into steps, identify dependencies and risks.
- Grounded answer: answer only from supplied context and cite source IDs.
- Tool router: choose among allowed tools using explicit criteria.

Anti-patterns: vague "be smart" instructions, mixing user data with hidden rules, asking for JSON but not validating it, allowing the model to invent missing fields, and changing many prompt variables at once without evals.

Prompt review checklist: Is the task singular and testable? Are inputs clearly delimited? Are missing facts handled explicitly? Is the output machine-validated? Are examples representative of edge cases? Could user or retrieved text override policy? Is the prompt versioned and covered by an eval dataset?

Optimization method: establish a baseline, categorize failures, change one variable, rerun the same evals, compare quality/cost/latency, and keep the change only if the tradeoff is acceptable.

Current OpenAI development centers on the Responses API for new projects, with built-in and custom tools available through the platform. The Agents SDK adds orchestration primitives such as agents, tools, handoffs, sessions, tracing and human-in-the-loop patterns.

Core integration checklist:

- server-side API key
- explicit model/config selection
- instructions and input separation
- timeouts/retries
- structured outputs or tool schemas
- usage/cost tracking
- trace/request IDs
- validation before side effects
- eval suite before release

Typical request lifecycle: validate input -> build instructions/context -> call Responses API -> inspect structured/model output -> execute allowed tools if required -> validate tool results -> generate final response -> record usage/trace -> return safe application output.

Never assume model names, context limits, prices or deprecation dates are permanent. Confirm official developer documentation during project kickoff and again before production release.

Official docs: <https://developers.openai.com/api/reference/responses/overview/> | <https://developers.openai.com/api/docs/guides/function-calling> | <https://developers.openai.com/api/docs/guides/tools> | <https://openai.github.io/openai-agents-python/>

Recommended workflow shape: Trigger -> Authenticate/Validate -> Normalize -> Deduplicate -> AI/Rules -> Branch -> Action -> Verify -> Log -> Notify.

Patterns to reuse:

- sub-workflow for error handling
- idempotency key before create/update operations
- retry only retryable errors
- pagination loop with rate-limit awareness
- human review queue for low confidence
- separate credentials by environment
- workflow JSON in version control
- correlation ID propagated across nodes
- test fixtures for webhook payloads

For AI workflows, keep deterministic steps outside the AI node whenever practical. An LLM should interpret ambiguity; it should not replace simple validation, arithmetic, permissions or routing rules that code can perform reliably.

Production review: confirm webhook authentication, idempotency, explicit timeouts, 429/5xx behavior, pagination, credential ownership, workflow export/versioning, test payloads, alert destinations and recovery procedure. For each write action, document exactly which credential performs it and what scope that credential has.

Official docs starting point: <https://docs.n8n.io/build/integrate-ai/>

Ingestion: source ownership, parser quality, chunking, metadata, version, hash, delete/update behavior.

Retrieval: query handling, top-k, metadata/permission filters, hybrid search/reranking if needed.

Generation: grounded instruction, citation format, abstention policy, context limits.

Evaluation: answerable/unanswerable questions, retrieval recall, groundedness, citation accuracy, latency and cost.

Security: access control before retrieval, tenant isolation, prompt injection from documents, sensitive-source handling.

Operations: sync schedule, failed ingestion alerts, stale index monitoring, source deletion, re-embedding plan when models/index configuration changes.

Evaluation set design: include direct-answer questions, multi-document questions, near-duplicate concepts, stale-policy traps, ambiguous requests, permission-denied sources and questions with no answer. Record expected evidence IDs, not only expected prose.

Cost controls: avoid embedding unchanged content repeatedly; hash/version sources, batch ingestion where supported, tune chunking from evidence, and measure retrieval quality before increasing top-k/context size.

A RAG project is complete only when data lifecycle, permissions, evaluation and source traceability are engineered.

Define goal and stop condition. List every allowed tool. Separate read vs write tools. Define tool input schemas and validation. Define memory/state fields. Set turn/tool/cost limits. Add policy checks. Add human approval before high-impact actions. Persist state if work may pause or resume. Add traces. Create eval cases for routing, tool selection, argument correctness and completion.

Prefer workflow logic when the sequence is known. Add agent autonomy only where the next step genuinely depends on uncertain context. Multi-agent systems should have a concrete reason such as specialization, permissions or distinct instruction sets; more agents is not automatically more capable.

Agent acceptance tests should include: correct tool selected, wrong tool rejected, arguments valid, permissions respected, max-turn behavior, human approval interruption/resume, duplicate side-effect prevention, recovery after tool error, and final answer supported by tool results.

Operationally, track turns, tool calls, usage/cost, tool-error rate, human escalation rate and task-success score.

Secrets: server-side only, scoped credentials, rotation plan.

Data: minimize collection, classify sensitivity, define retention, encrypt in transit/at rest where applicable.

Prompt injection: treat external text as untrusted data; never let retrieved content redefine permissions.

Tools: least privilege, allowlists, validation, approval for destructive/financial/external communication actions.

Tenancy: isolate customer data and retrieval permissions.

Logging: avoid unnecessary PII/secrets; protect traces.

Supply chain: pin/review dependencies and workflow imports.

Human factors: clearly label AI-generated drafts where needed and provide escalation.

Testing: adversarial prompts, malformed inputs, oversized payloads, tool misuse, data exfiltration attempts and duplicate events.

Security review order: identify assets -> identify trust boundaries -> list untrusted inputs -> list tools/credentials -> rank impact of misuse -> add preventive controls -> add detection/alerts -> test abuse cases -> define response/recovery.

For client work, include a credential ownership table showing provider, purpose, scope, storage location, owner, rotation method and production transfer status.

Business: What outcome matters? Who performs the process today? How often? How long does it take? What does an error cost?

Process: What triggers work? What are the steps, decisions and exceptions?

Systems: Which apps, APIs, files and databases are involved? Who owns credentials?

Data: What formats, volumes, sensitive fields and retention rules apply?

AI: Which steps need interpretation vs deterministic logic? What mistakes are acceptable?

Actions: Can the system send, delete, update, approve, charge or change permissions? What needs human approval?

Acceptance: What measurable result defines success?

Operations: Who monitors it? What happens on failure? What support window is required?

Commercial: deadline, budget, milestones, ownership, documentation and maintenance expectations.

Discovery output should be written: current-state process, future-state process, data-flow diagram, assumptions, risks, API dependencies, acceptance tests, milestone plan and out-of-scope list. Ask the client to confirm these before substantial implementation.

Useful final question: If this automation is wrong at 2 a.m., what should happen? The answer often reveals the true escalation and reliability requirement.

Short technical proposal pattern:

"You need [outcome], not just an AI demo. I would build Phase 1 as [trigger -> core logic -> system action], keep [high-risk action] behind approval, and measure success with [metric]. The main dependency I would confirm first is [API/data constraint]. I have a relevant example in [portfolio project]. First milestone: [small deliverable]. Questions: [2-3 sharp questions]."

Avoid generic openings such as "I am an expert with 10 years experience" when the client gave enough information to discuss the problem directly. Strong proposals show that you understood the workflow, can identify risk, and can suggest a small measurable first milestone.

Proposal quality test: remove your name and profile. Does the proposal still prove that you understood this specific job? If it could be pasted unchanged into 50 unrelated postings, it is too generic.

For fixed-price work, define milestone outputs and acceptance conditions. For exploratory work with unknown APIs/data quality, prefer a paid discovery or prototype milestone before committing to a large fixed scope.

A strong case study contains: problem, users, manual process, architecture, implementation details, security/reliability controls, evals/tests, demo, measurable result, limitations and next phase.

Interview preparation categories:

1. LLM/prompt fundamentals
2. APIs/function calling
3. n8n reliability
- 4. RAG**
5. agents/orchestration
6. security/privacy
7. deployment/observability
8. system design
9. client discovery
10. debugging scenario

Practice explaining tradeoffs, not memorizing definitions. A senior answer usually states assumptions, chooses an architecture, identifies risks, explains validation and describes how success will be measured.

Portfolio scoring: clarity of business problem 10, architecture 15, implementation 20, reliability 15, security 10, evals/testing 15, documentation 10, demo quality 5. A smaller project with strong evidence beats a large project that cannot be reproduced.

Practice a 5-minute system design, 10-minute debugging scenario and 15-minute client discovery simulation every week.

Official references checked for this edition (August 2026): OpenAI Developers - Responses API, Function Calling, Embeddings, Tools, Agent Evals; OpenAI Agents SDK docs; n8n AI integration and AI Agent docs; LangChain/LangGraph official documentation.

90-day roadmap:

Days 1-30: finish all labs; strengthen Python/TypeScript, REST, Git and SQL.

Days 31-60: rebuild two projects without following the handbook; add evals, Docker and production logging.

Days 61-90: choose a niche (support automation, sales ops, RAG, document automation, voice, or agent systems), publish 3 polished case studies, run weekly mock discovery/interviews, and analyze real outsourcing briefs for gaps.

Reference URLs:

<https://developers.openai.com/api/reference/responses/overview/>

<https://developers.openai.com/api/docs/guides/function-calling>

<https://developers.openai.com/api/docs/guides/embeddings>

<https://developers.openai.com/api/docs/guides/agent-evals>

<https://openai.github.io/openai-agents-python/>

<https://docs.n8n.io/build/integrate-ai/>

<https://docs.langchain.com/oss/python/langgraph/overview>

Technology changes quickly. Re-validate model availability, SDK syntax, pricing, limits and platform features before every client production release. Quarterly, re-run portfolio projects against current SDKs and refresh demos.